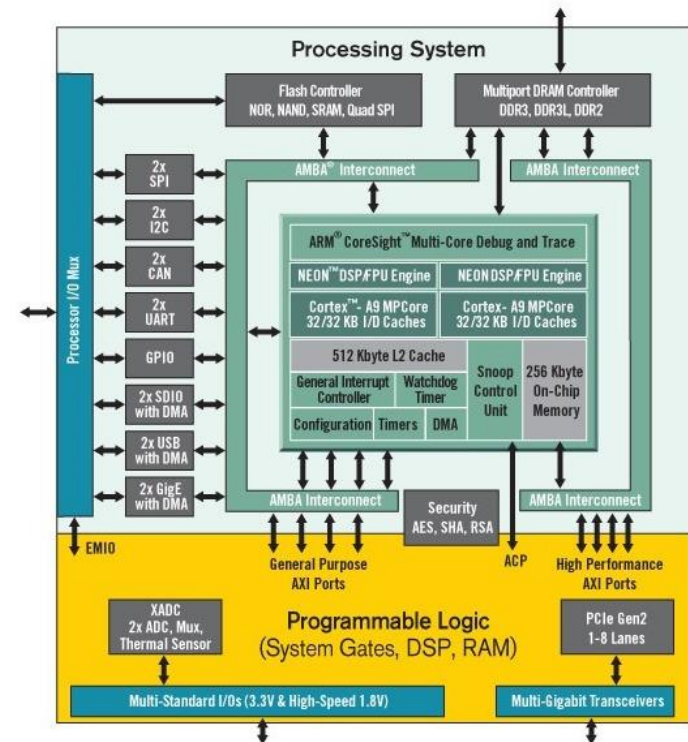
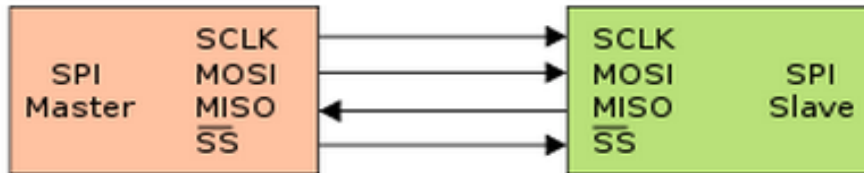
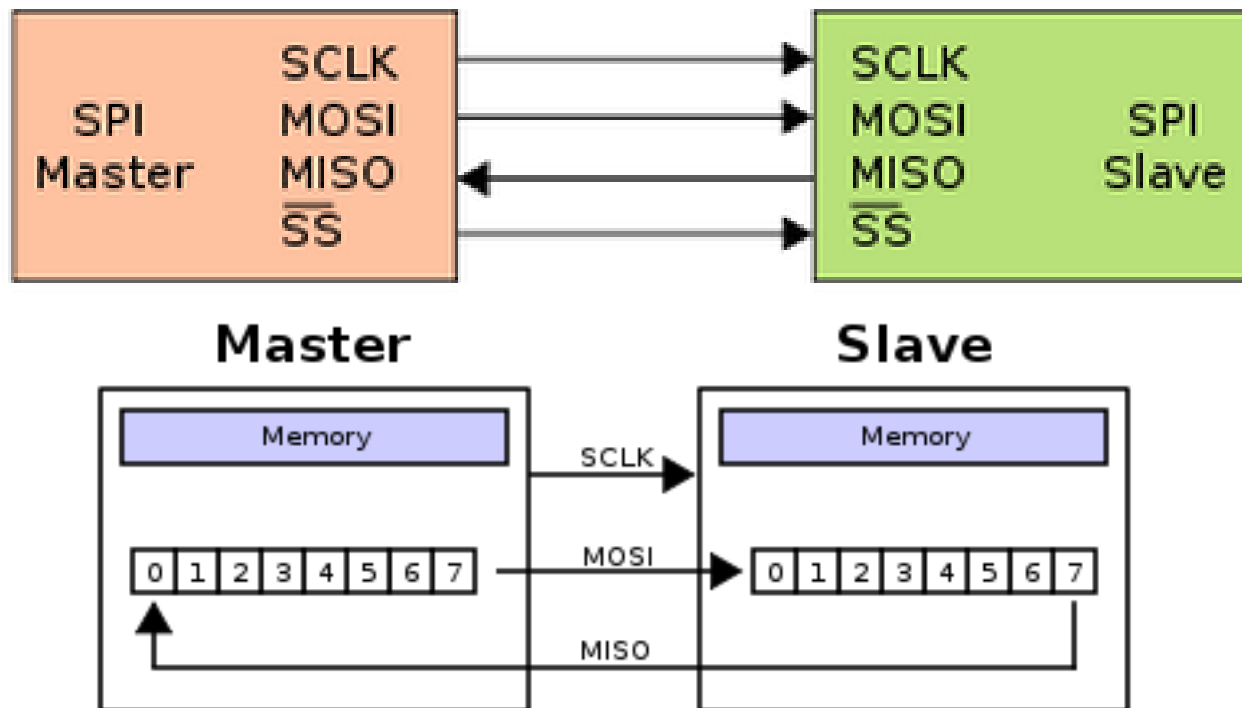


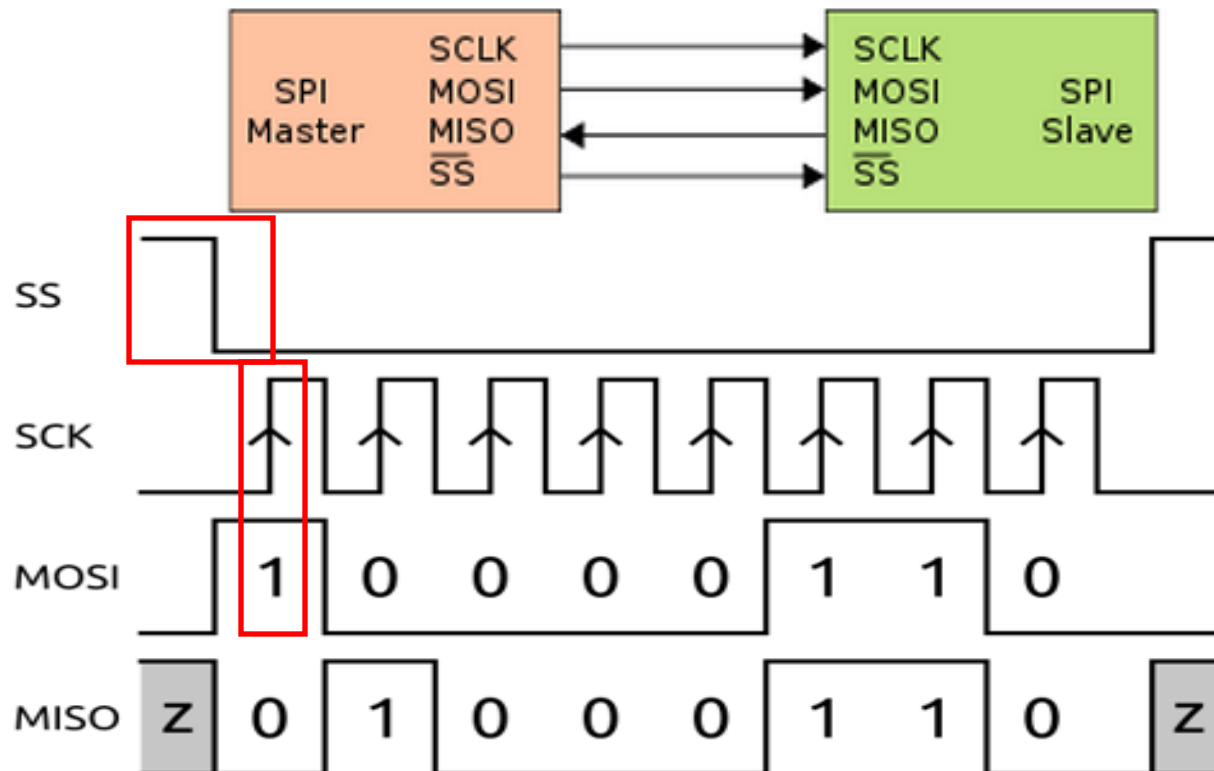
Zynq SPI Peripherals: ADC and DAC



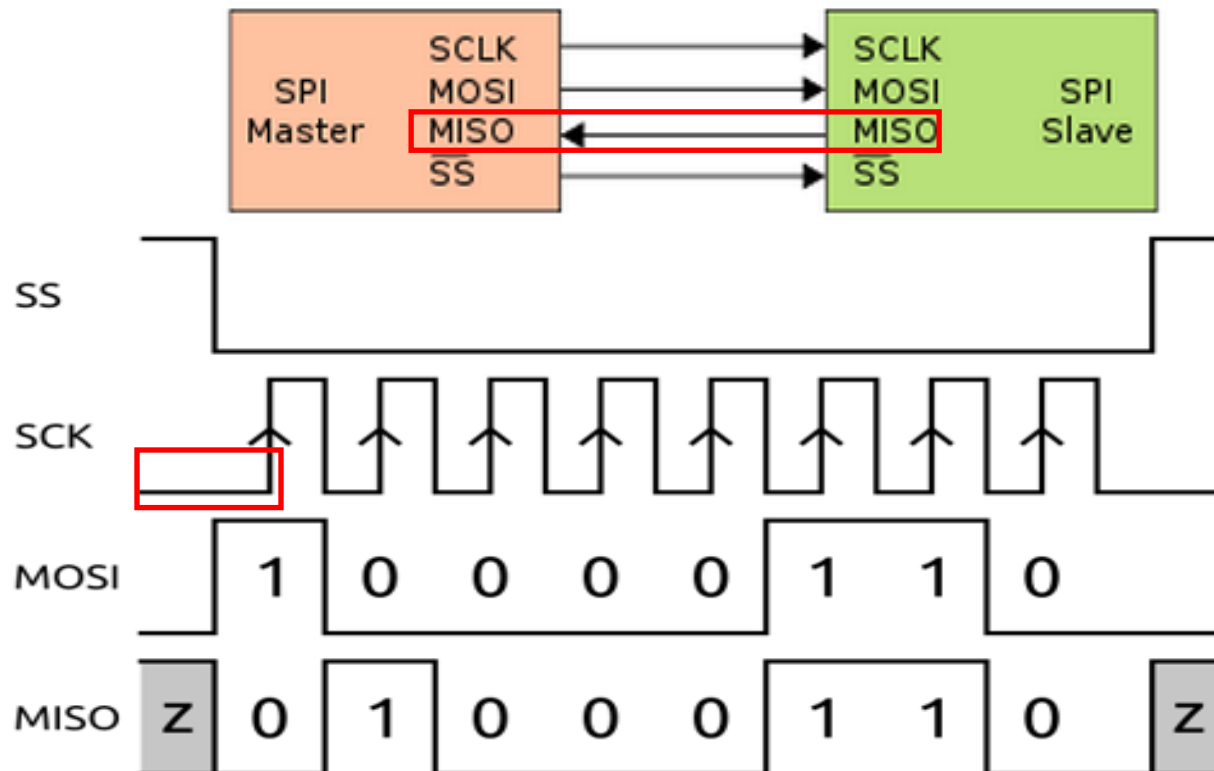
SPI Bus. The *Serial Peripheral Interface (SPI)* consists of the serial clock *SCLK*, master output/slave input *MOSI*, master input/slave output *MISO* and slave select *SS* signals.



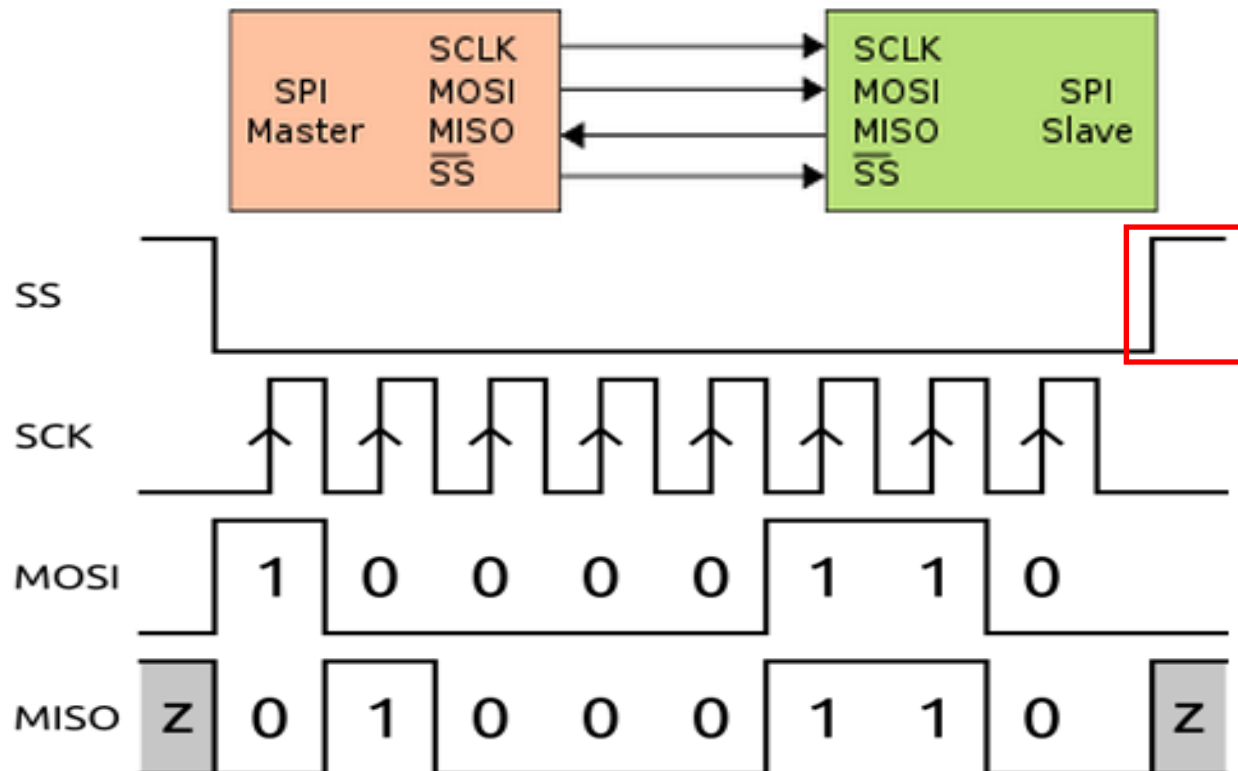
SPI Bus. The SPI bus serial data transmission begins with the SS signal at logic 0. Either the *rising* (as shown) or *falling* edge of SCLK as specified by the slave device signifies valid MOSI data with the *most significant bit* (MSB) sent first.



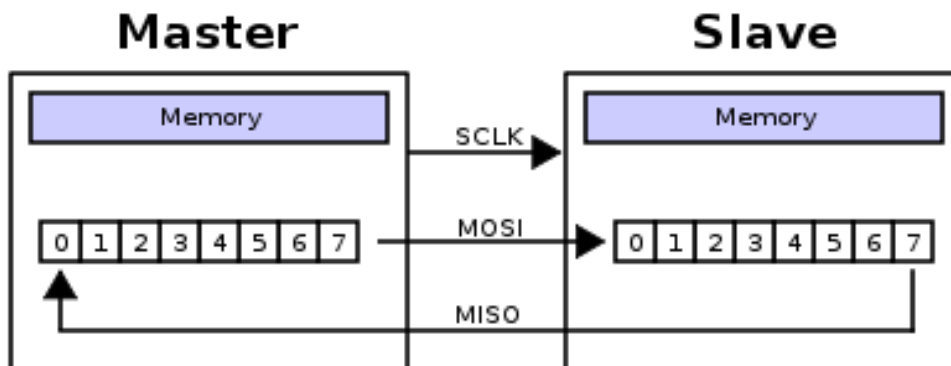
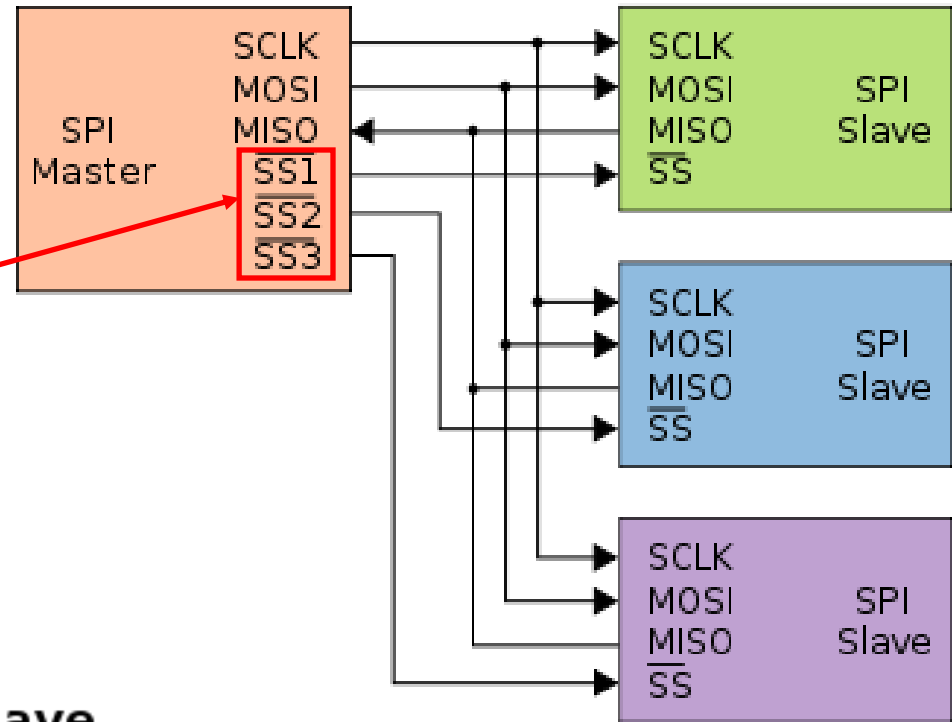
SPI Bus. The default value of the SCLK signal for a rising edge valid data transmission is logic 0 and for a falling edge it is logic 1. Concurrently, the MISO signal outputs the MSB of the *last* (not the current) data transmission to the SPI bus master device.



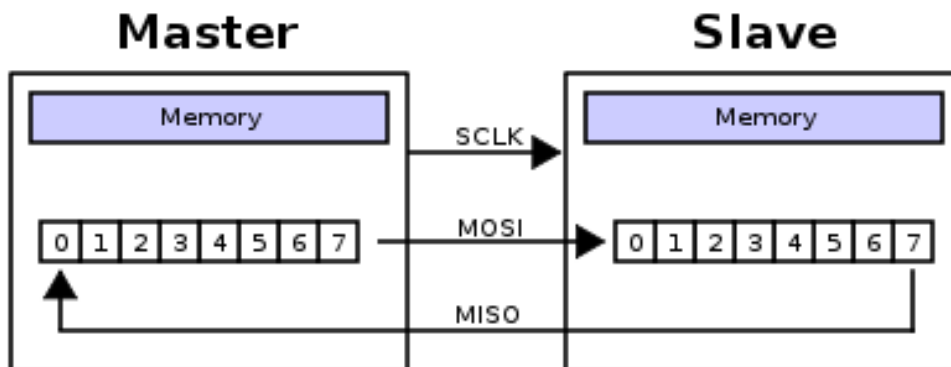
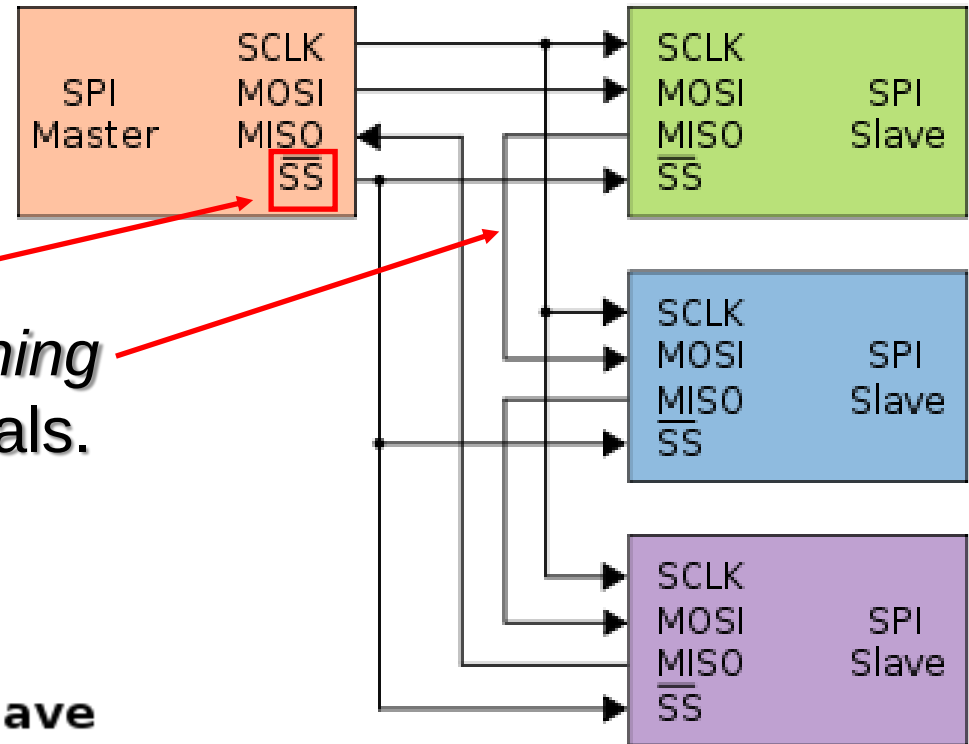
SPI Bus. This serial data transmission continues with any number of SCLK cycles until the SS signal returns to logic 1.



SPI Bus. Multiple independent SPI slave devices can be interfaced to a single SPI master with *multiple* SS signals.



SPI Bus. Alternatively, Multiple cooperative SPI slave devices can be interfaced to a single SPI master with a *single* SS signal and *daisy chaining* the MISO and MOSI signals.



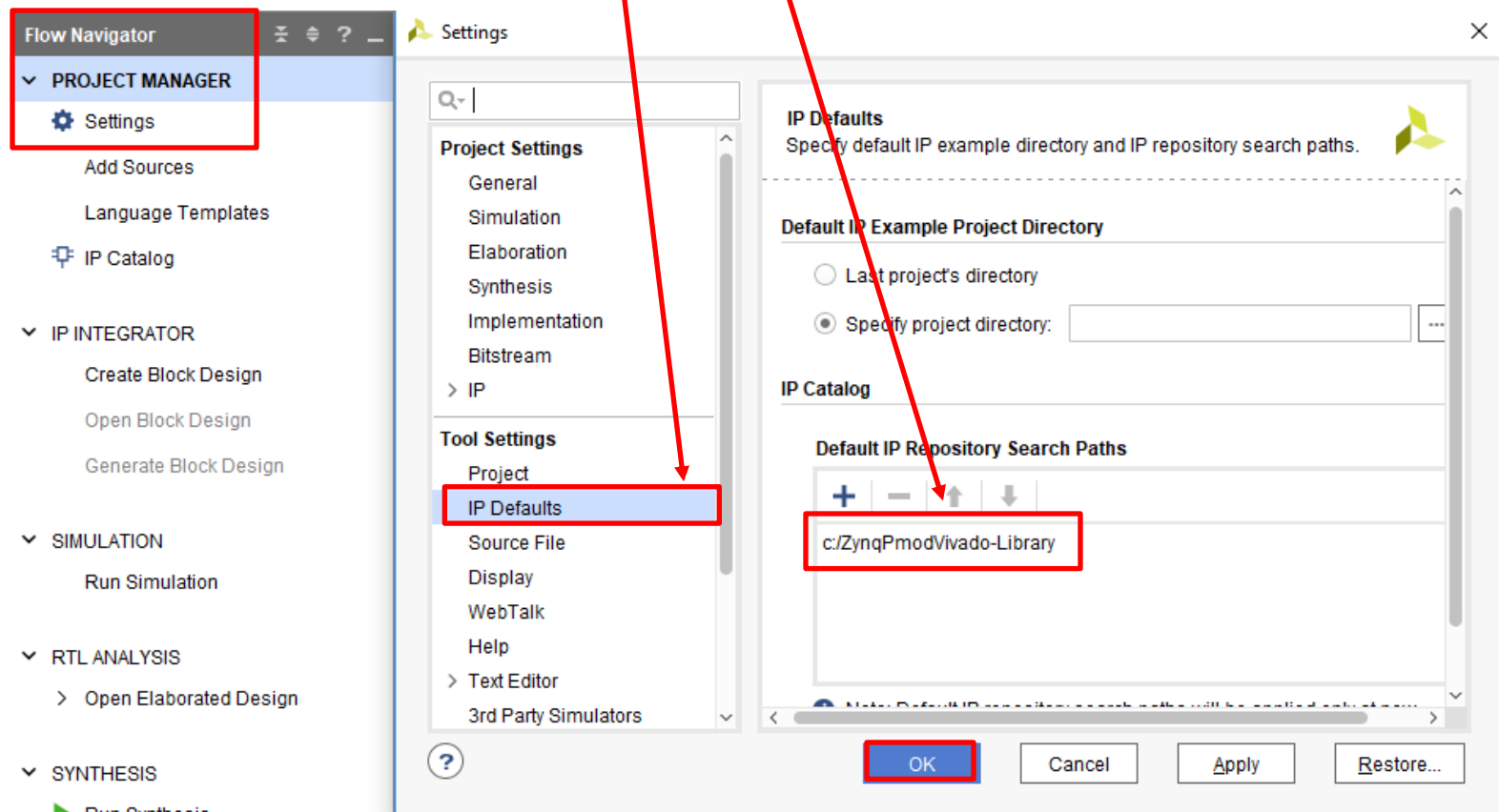
Zynq SPI Peripherals – Vivado Library. Download *ZynqPmodVivado-Library.zip* file from the ECE3623 Canvas and extract the file to a new folder *C:/ZynqPmodVivado-Library*.



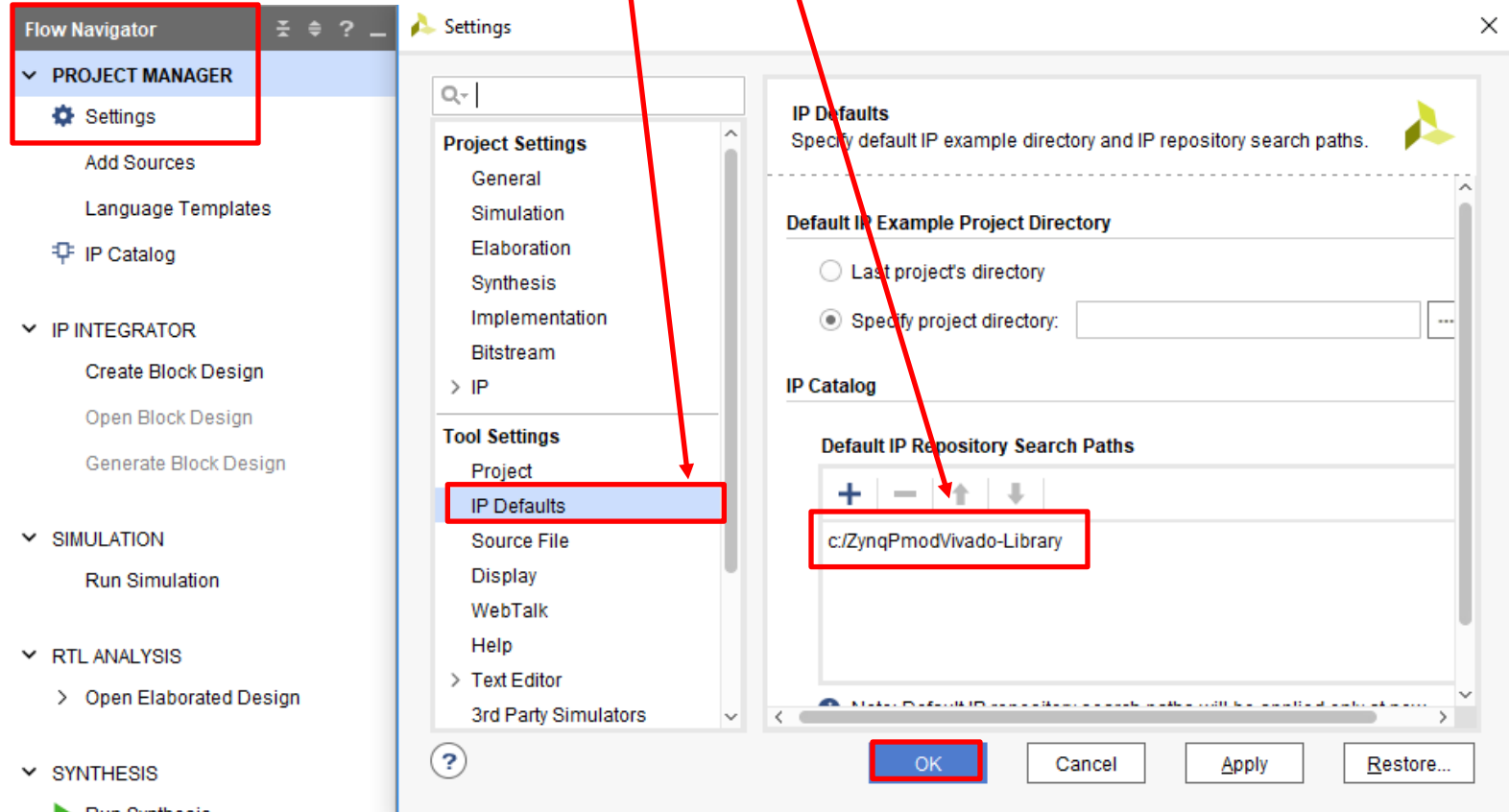
Create a dummy Vivado project as *project_1* with no sources to add the additional IP library.



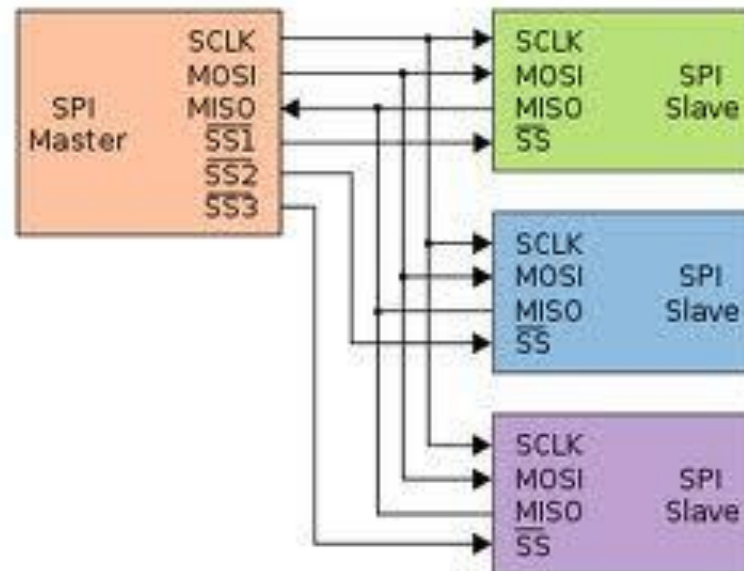
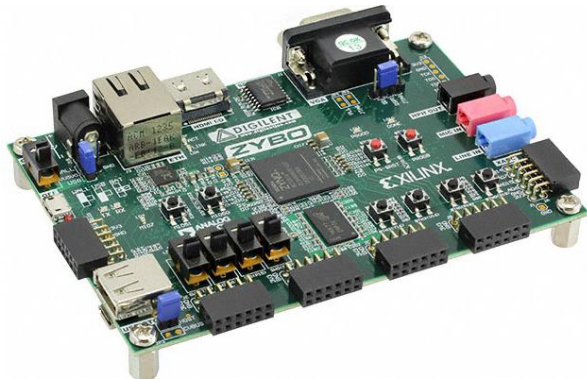
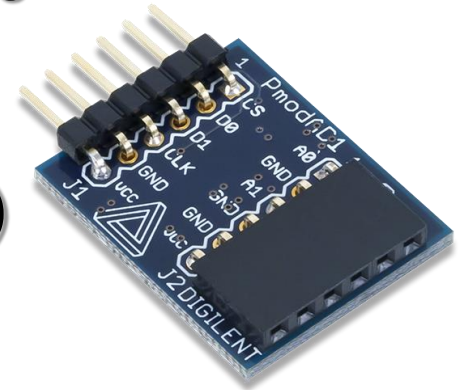
Zynq SPI Peripherals – Vivado Library. In the *Flow Navigator...Project Manager* select *Settings* then *IP Defaults*. Add *C:/ZynqPmodVivado-Library* to the *Default IP Repository*. OK to continue.



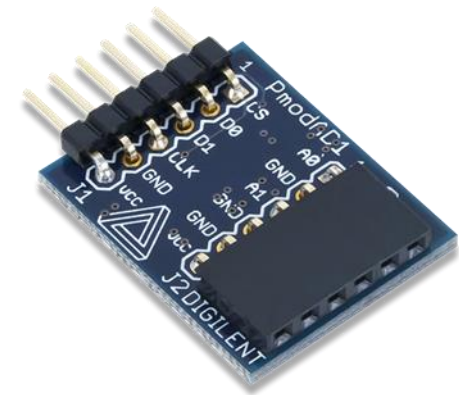
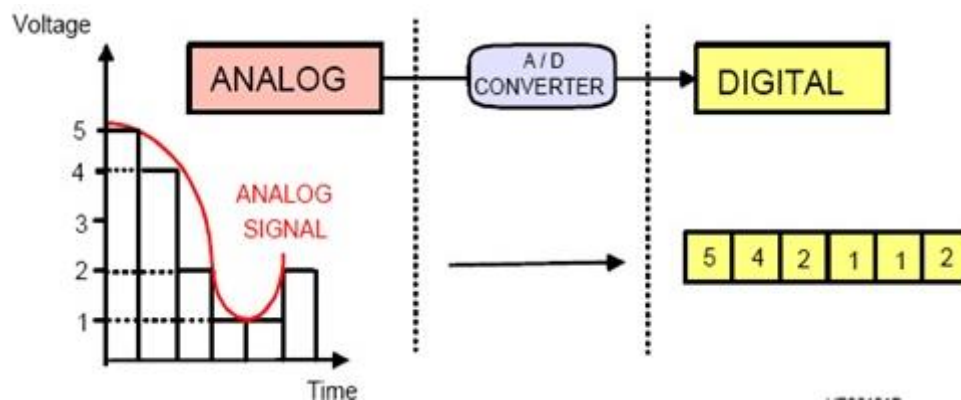
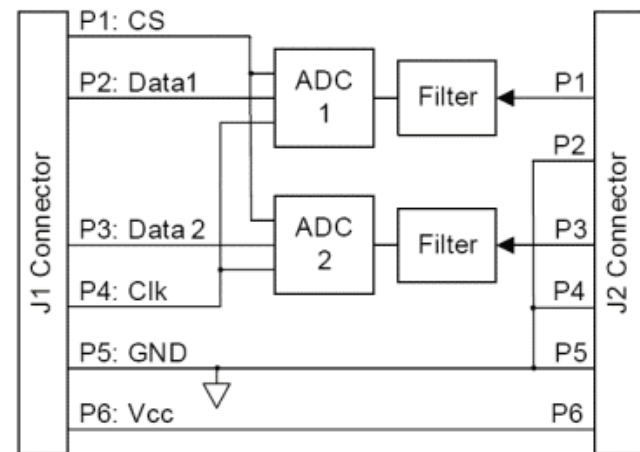
Zynq SPI Peripherals – Vivado Library. This installs the extended *ZynqPmodVivado-Library* to be used in new projects in Vivado.



Zynq SPI Peripheral - ADC. As an example of SPI bus interfacing to a Zynq device, a Digilent Pmod AD1 two-channel, 12-bit resolution analog-to-digital converter (ADC) is interfaced to the Zynq PS in the Vivado HLx design project using the extended *ZynqPmodVivado-Library*.



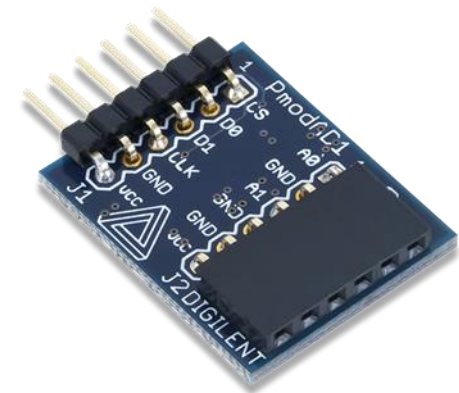
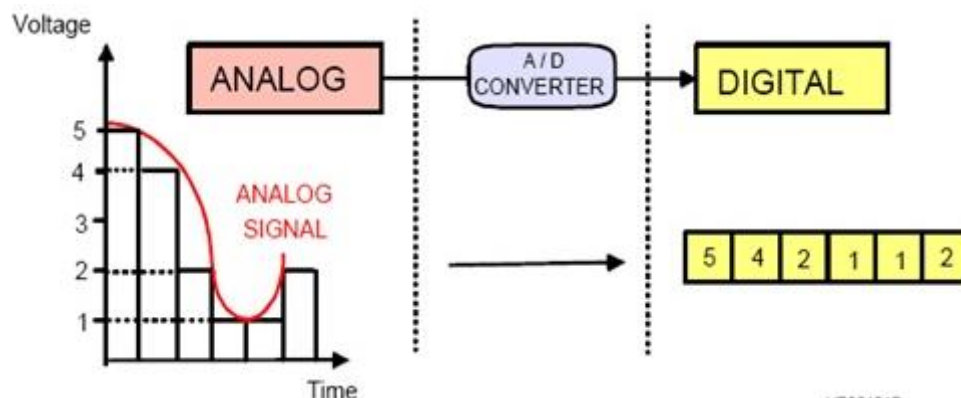
Zynq SPI Peripheral - ADC. An ADC provides a discrete, sampled binary data output signal from a continuous analog input signal for embedded system design in audio processing, analog and digital baseband and bandpass communication and digital process control.



Zynq SPI Peripheral - ADC. The general equation for an n-bit binary data output $D[n-1:0]$ for the ADC that inputs a unipolar (only positive amplitude) analog signal is:

$$D[n-1:0] = G \frac{V_{IN} - V_{REF}}{V_{FS}} 2^{n-1}$$

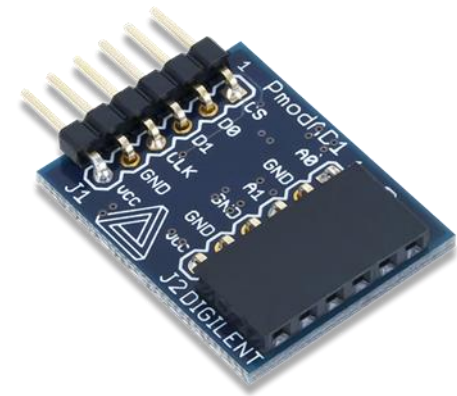
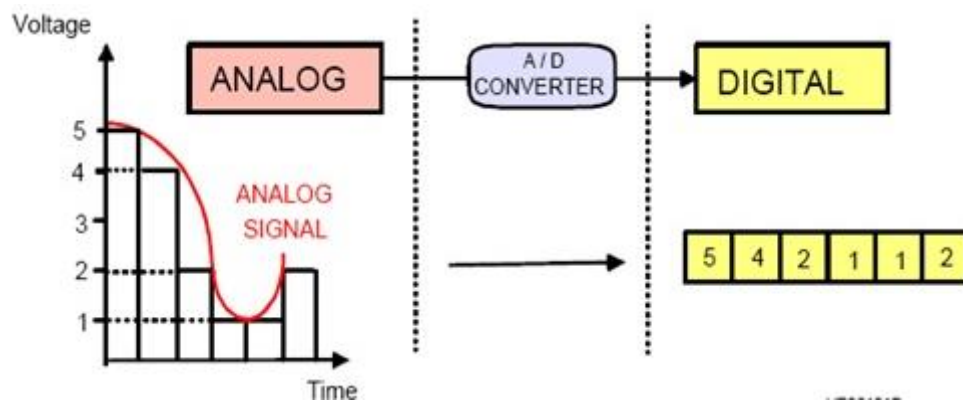
The maximum rate at which the ADC can sample an analog signal and produce n-bit binary data is its throughput rate R_{ADC} in samples per second.



Zynq SPI Peripheral - ADC. Since V_{IN} could be less than V_{REF} , the n -bit binary data output $D[n-1:0]$ is then in *two's complement* format with the most significant bit (MSB) representing the sign bit and a number whose integer value is between -2^{n-1} and $2^{n-1} - 1$

For example if $n = 12$, then the range is -2048 to 2047 .

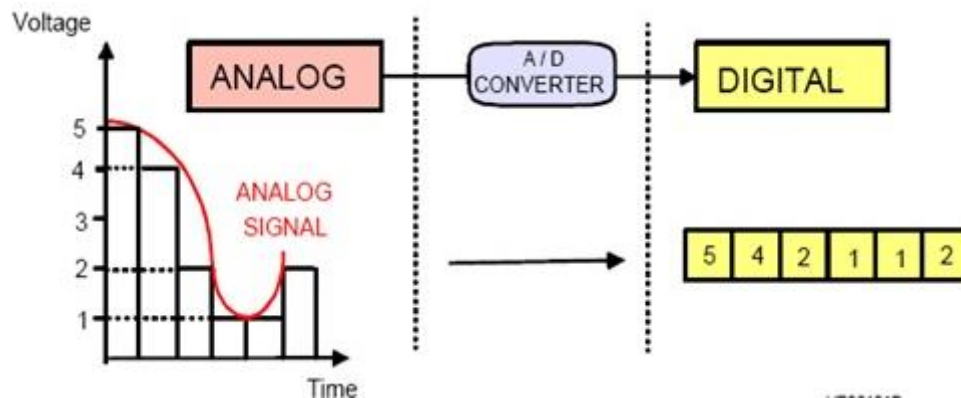
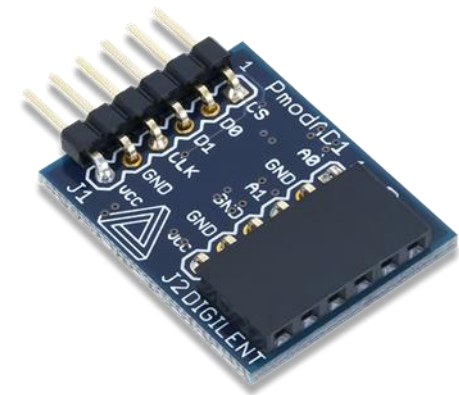
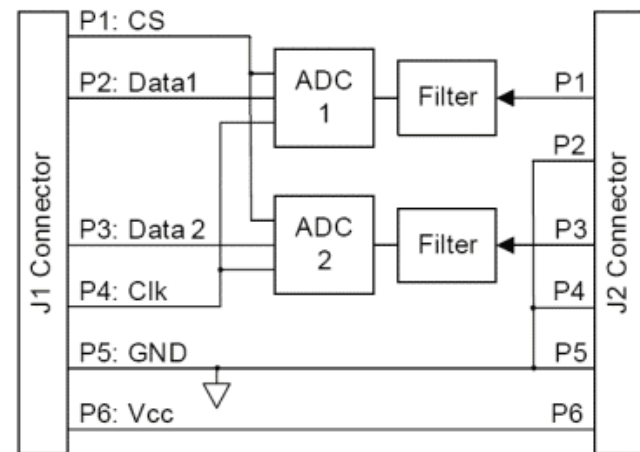
$$D[n-1:0] = G \frac{V_{IN} - V_{REF}}{V_{FS}} 2^{n-1}$$



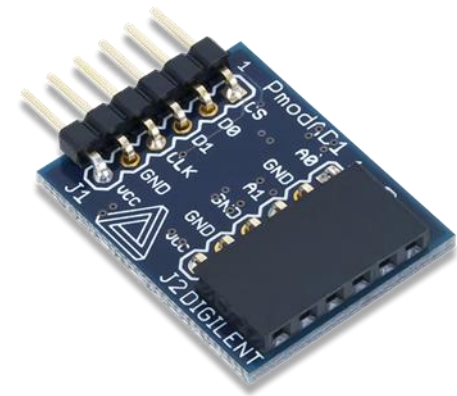
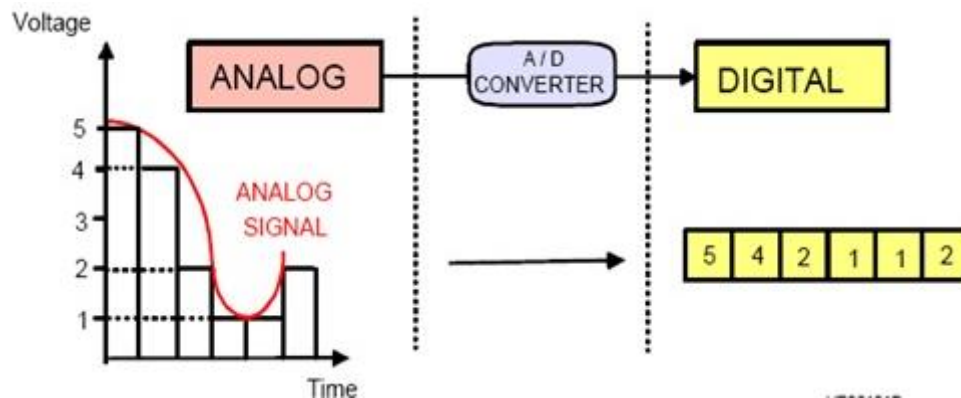
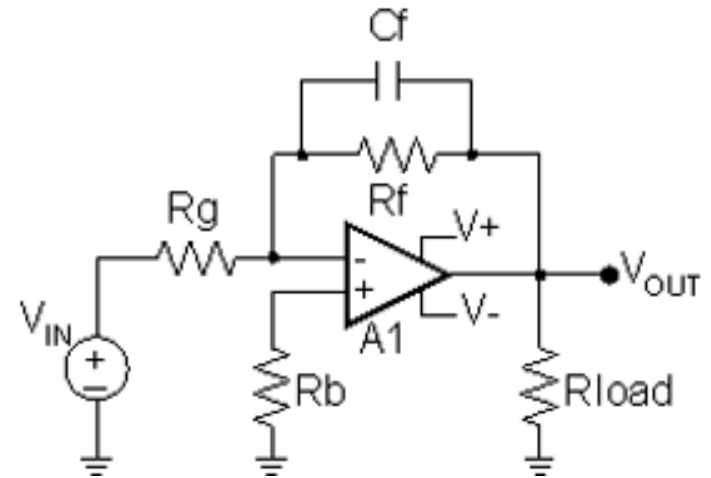
Zynq SPI Peripheral - ADC. However, the Pmod AD1 ADC uses $G = 1$, $V_{REF} = 0$, $V_{FS} = V_{DD} = 3.3\text{ V}$ and $n = 12$ for a digital range of 0 to 4095 for an input voltage for 0 to 3.3 V.

$$D[n-1:0] = G \frac{V_{IN} - V_{REF}}{V_{FS}} 2^{n-1}$$

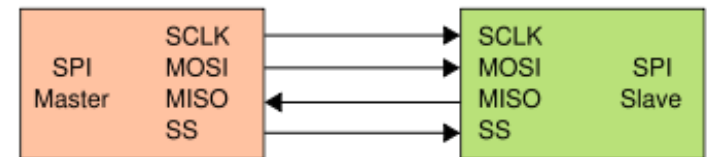
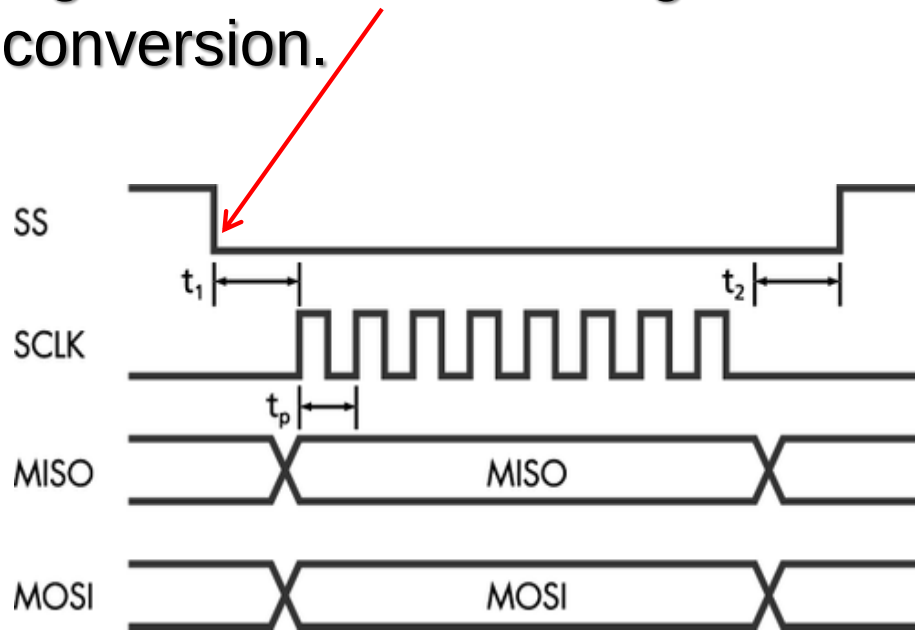
The PmodAD1 has two TI ADCS7476 12-bit ADCs.



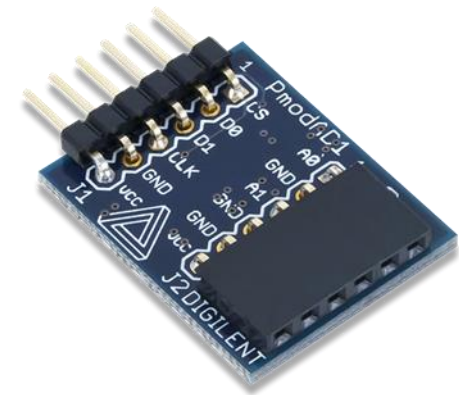
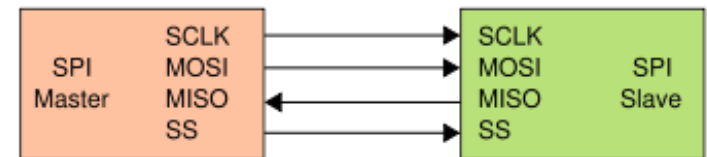
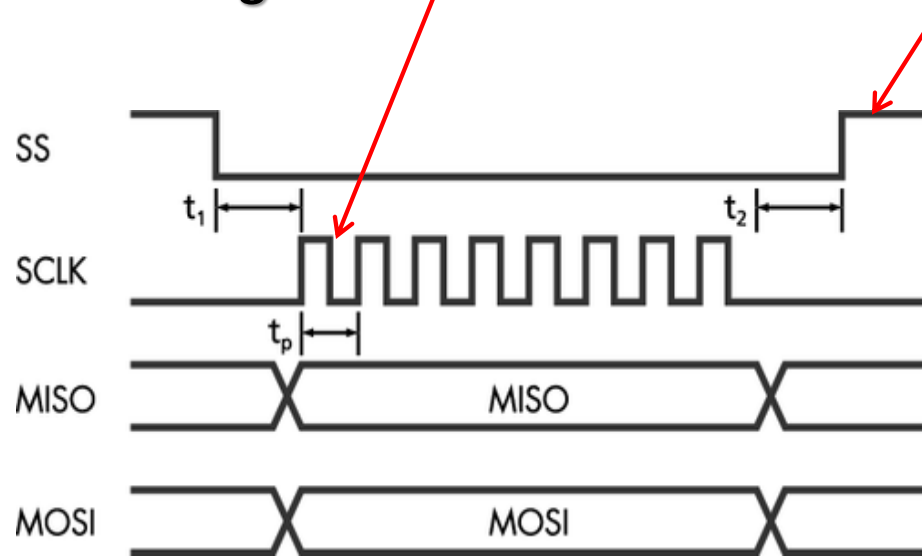
Zynq SPI Peripheral - ADC. An external analog preamplifier can also support the function of the ADC in an embedded design by providing a differential to single-ended signal interface, significant gain for an analog transducer or an *anti-aliasing* filter for discrete data sampling.



Zynq SPI Peripheral - ADC. The ADCs use an active low ADCCS signal and an SPI bus clock signal SCLK and output two data signals ADCD0 and ADCD1. The ADC SPI data communication protocol begins with the chip select signal ADCCS set to logic 0 which initiates analog data conversion.



Zynq SPI Peripheral - ADC. The Pmod AD1 ADCD7476 ADC uses the falling edge of the SPI clock signal SCLK to transmit a 16-bit data packet, with the first four bits as don't care bits and the most significant bit (MSB) of the data sent first. After all 16 bits have been sent, the chip select signal ADCCS is set to logic 1.



Zynq SPI Peripheral - ADC. Installation and use of the ADC is described in the *PmodAD1 and PmodDA2 Peripherals on Canvas.*

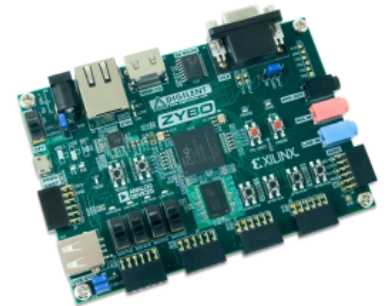


ECE3623 Embedded System Design Laboratory

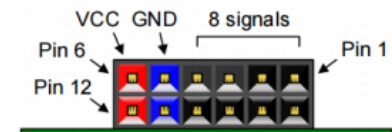
Dennis Silage, PhD
silage@temple.edu

PmodAD1 and PmodDA2 Peripherals

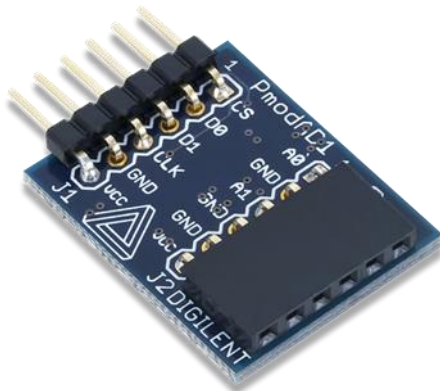
The Digilent Zybo Board does not feature an analog-to-digital converter (ADC) or digital-to-analog converter (DAC). The Digilent PmodAD1 and PmodDA2 provide this functionality and both 6-pin peripheral devices can be installed on the 12-pin Pmod jack on the lower left side of the Zybo Board.



The leftmost, front Pmod jack (J1) is available from the programmable logic (PL) of the Zynq SoC device and is shown as a *front view*. Note the location of pin 1 on the top right. Pin 7 is immediately below pin 1 on the bottom right. To facilitate the connection a 12 pin to two 6 pin cable is provided as shown in the Figure.

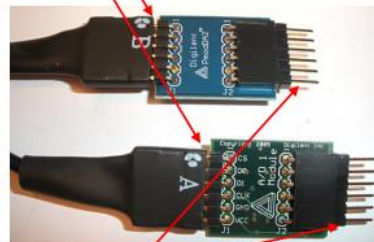
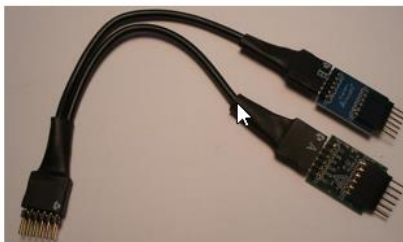


The PmodAD1 ADC is connected to the cable 6 pin connector A with the index mark positioned on pin 1 of J1 the SPI interface, as shown in the Pictures. The PmodDA2 is connected to the cable 6 pin connector B with the index mark position on pin 1 of J1 the SPI interface as shown in the Pictures.

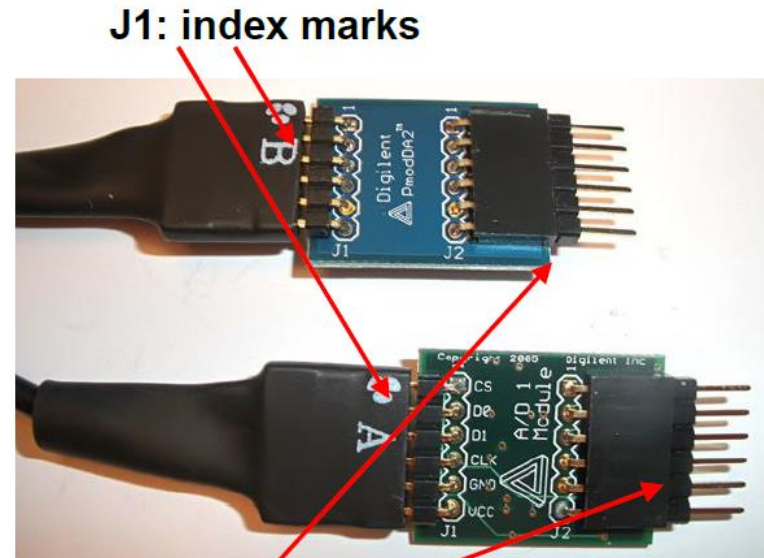
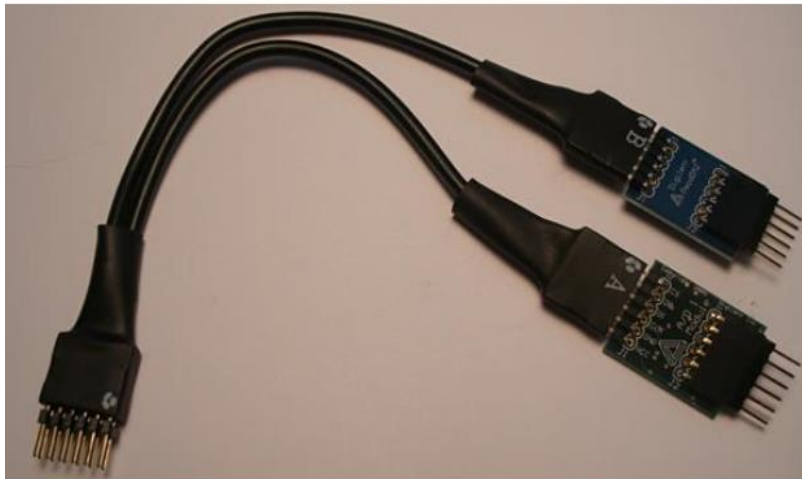


J1: index marks

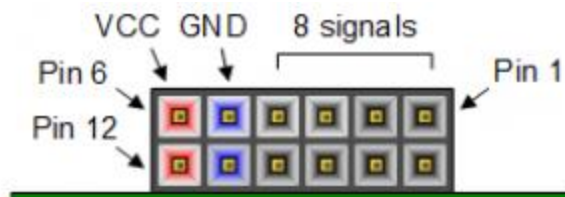
J2: 6 pin headers



Zynq SPI Peripheral - ADC. Installation and use of the ADC is described in the *PmodAD1* and *PmodDA2* Peripherals on Canvas.



J2: 6 pin headers input or output connectors



Zynq SPI Peripheral - ADC. Installation and use of the ADC is described in the *PmodAD1* and *PmodDA2* Peripherals on Canvas.

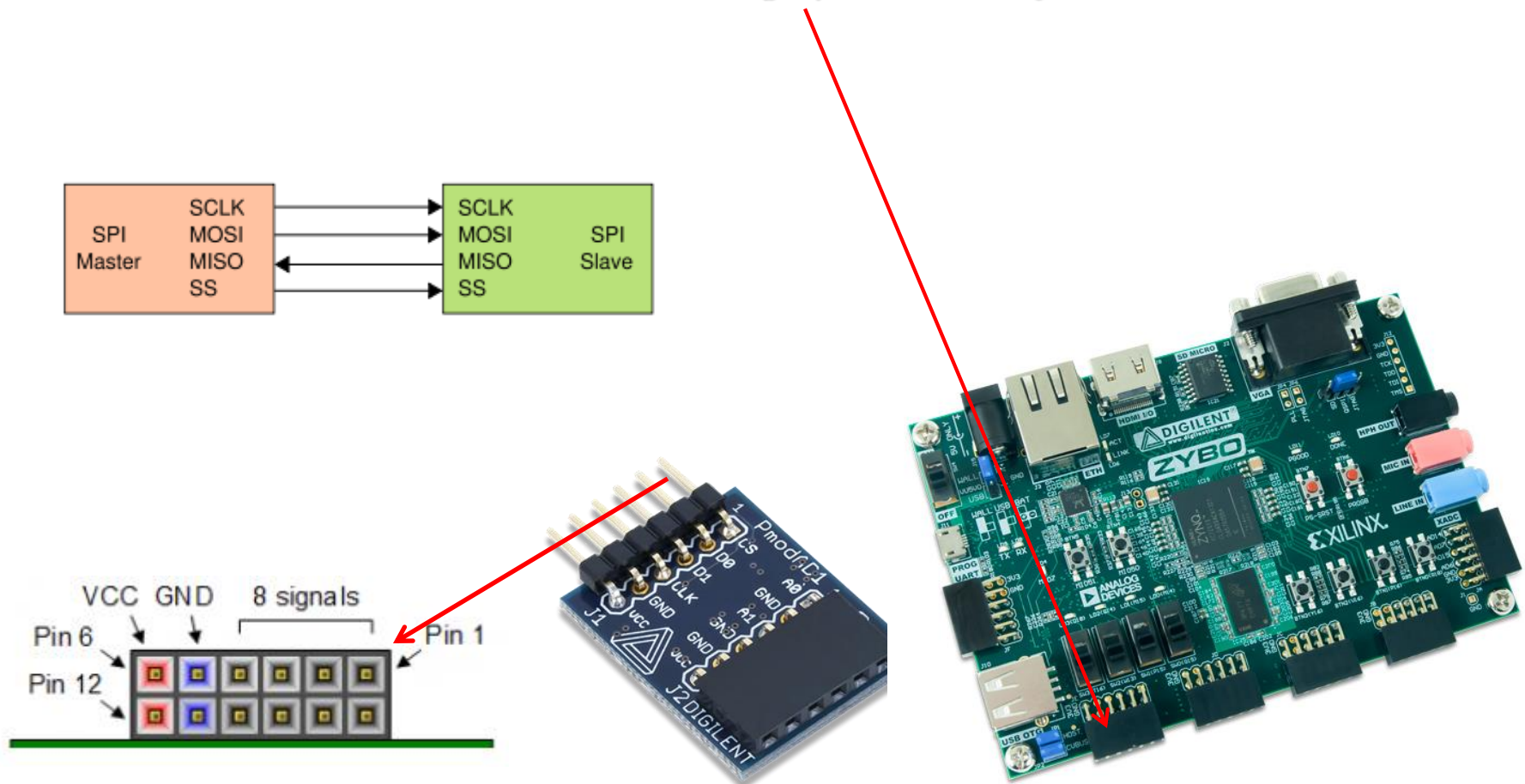
Zybo Board Pmod Jack

The 12 pin connector of the cable is provided with a 12 pin header which is inserted into the 12-pin Pmod jack on the lower left side of the Zybo Board (JE). The index mark shown in the picture **must be on the lower right side and not visible from the top surface** of the Zybo Board.

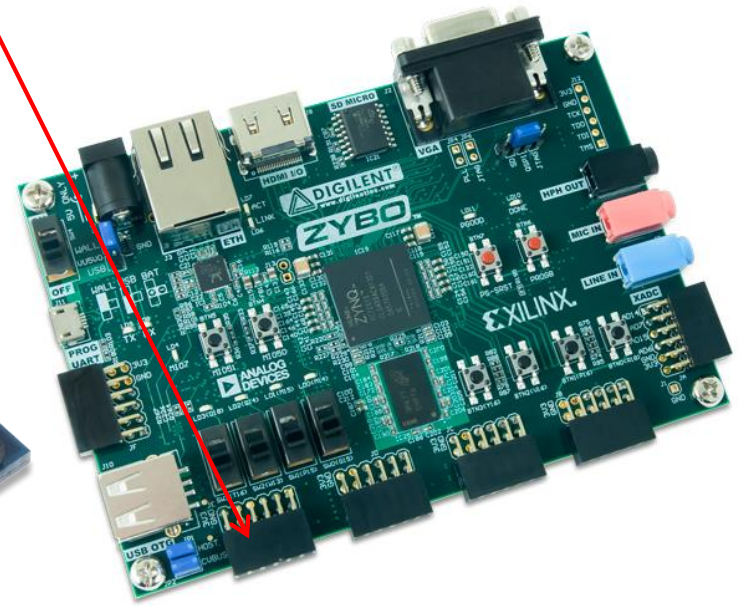
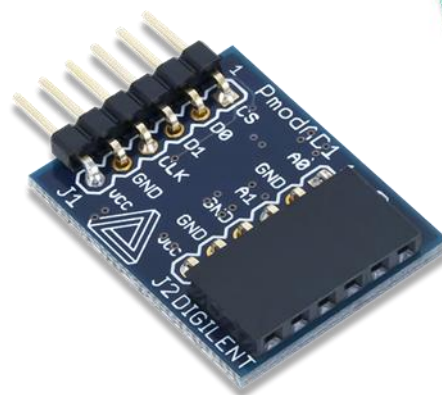
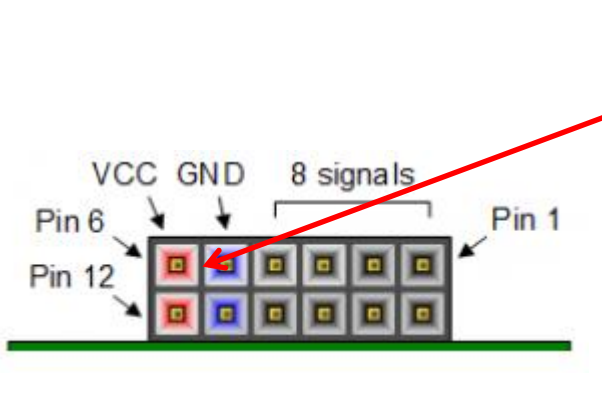
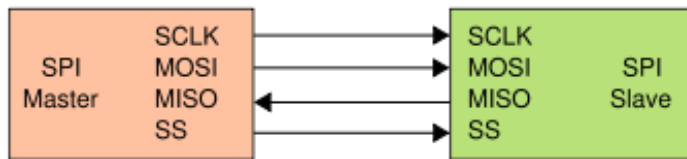
This insures that V_{CC} for the PmodAD1 and PmodDA2 are connected properly at pins 6 and 12 of the 12 pin Pmod connector of the Zybo Board. **Improper connection can damage the Zybo Board and the Pmod peripherals and is your responsibility.**



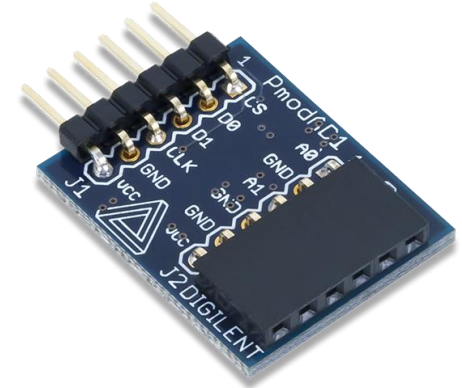
Zynq SPI Peripheral - ADC. The PmodAD1 ADC is connected to the *top 6 pins* of the 12-pin peripheral hardware module connector (je) of the Zybo Board.



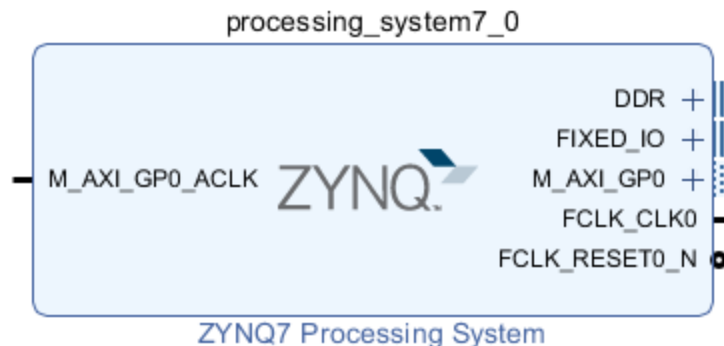
Zynq SPI Peripheral - ADC. The SYNC signal is pin je[1], the two output data signals ADCD0 and ADCD1 are pins je[2] and je[3] and the SPI bus clock signal SCLK is pin je[4]. Pins 5 and 6 are DC ground and power (VDD = +3.3 V DC).



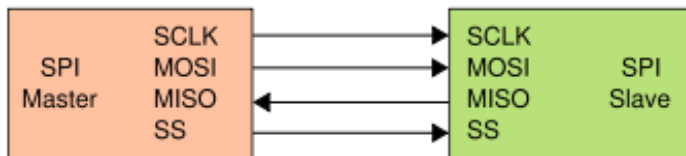
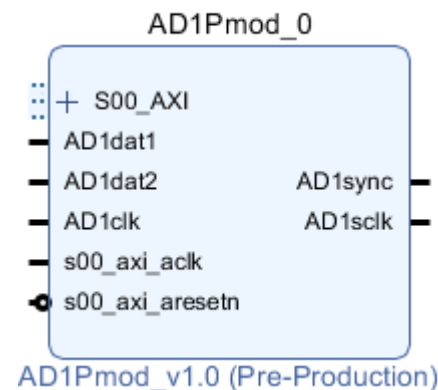
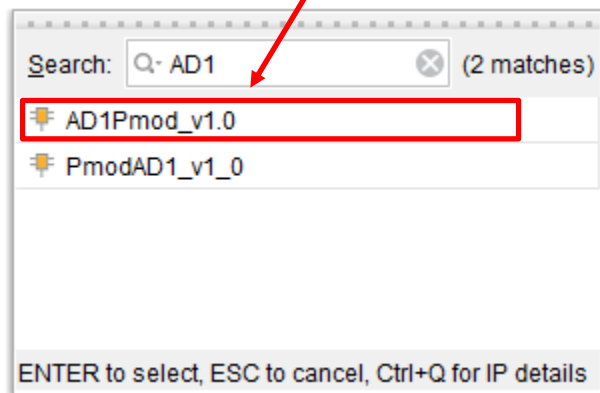
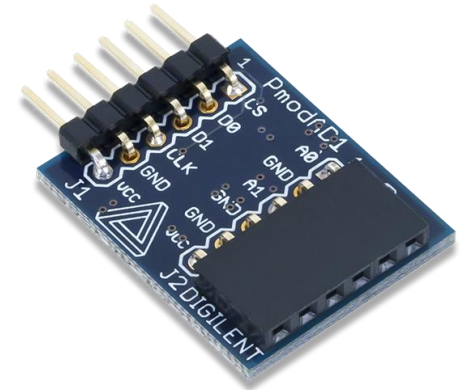
Zynq SPI Peripheral - ADC. As in the Zynq Book Tutorial Exercises, create a Vivado project as *AD1Pmod* in a new folder C:/zybo_Pmod_Projects.



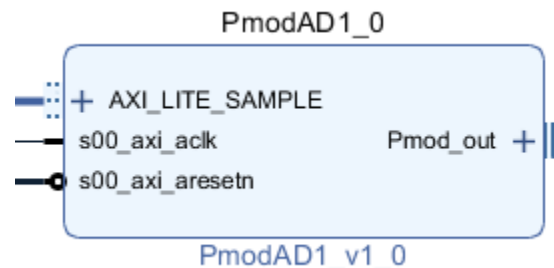
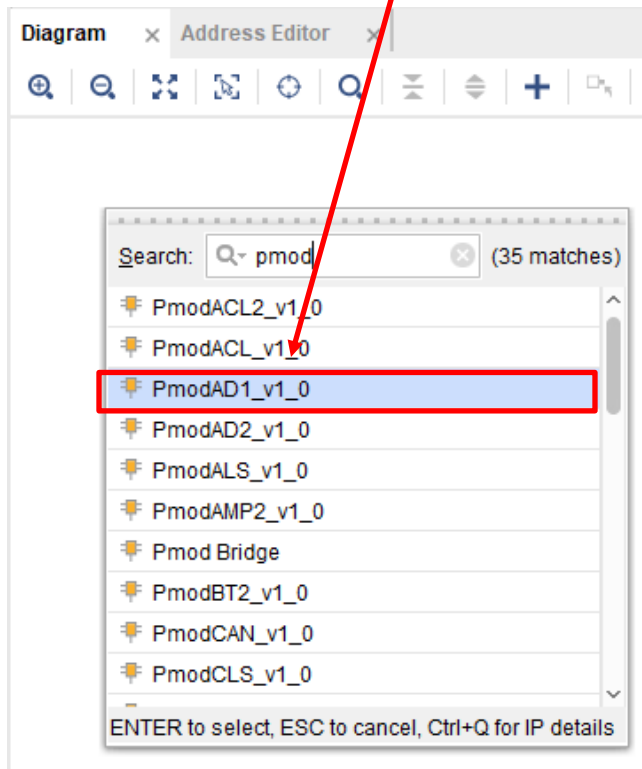
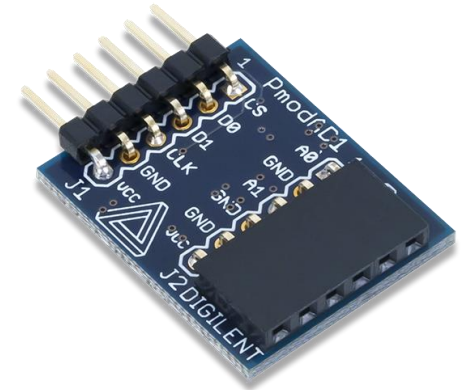
The project is *RTL* and has no source or constraint files. Select Zybo board and configure a basic Zynq processing system. In *Project Manager...Settings* the top module name is *design_1*.



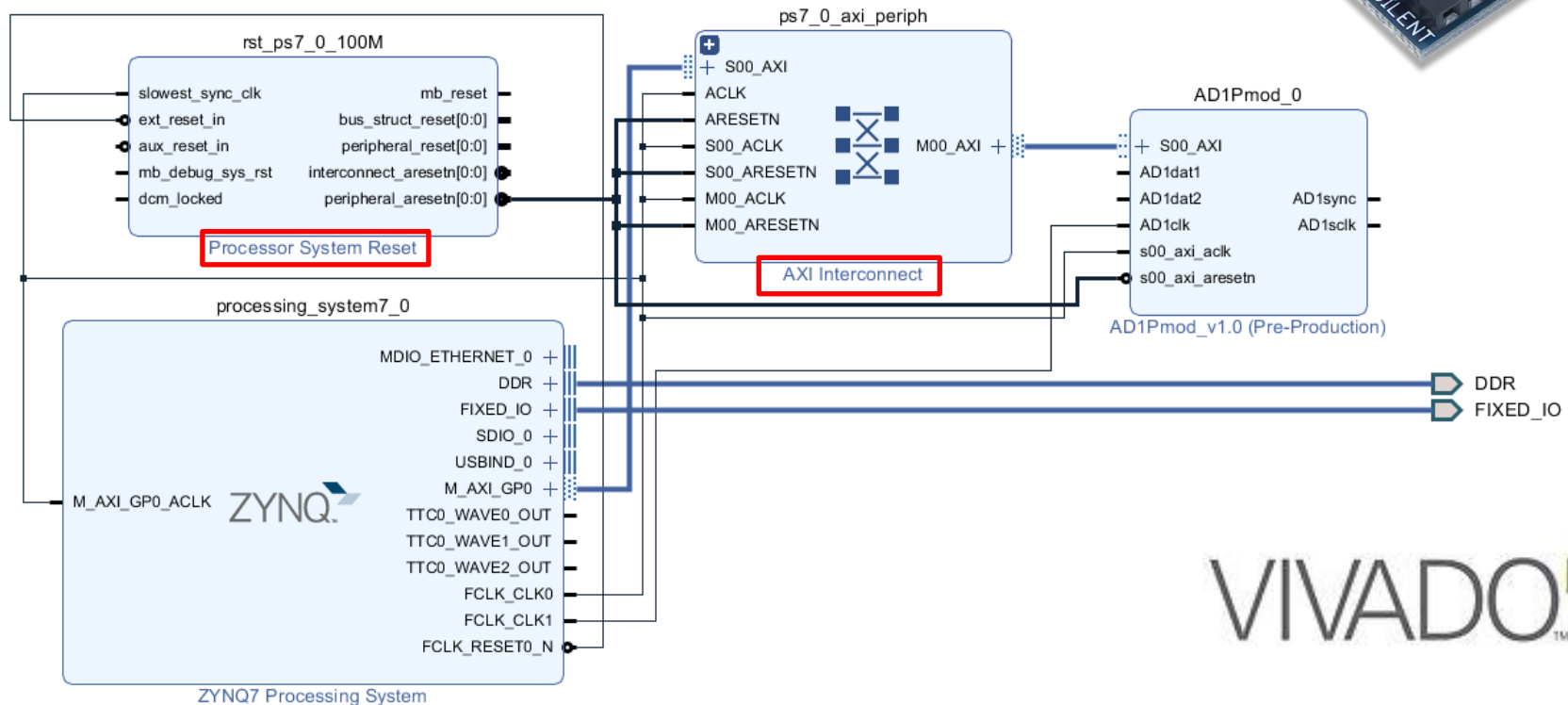
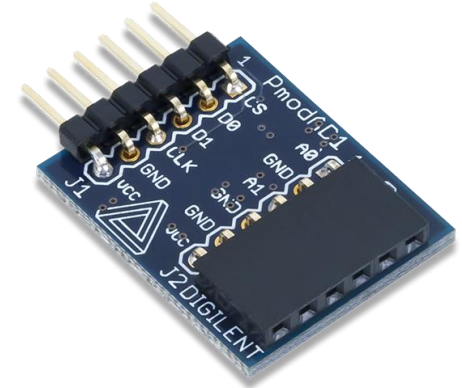
Zynq SPI Peripheral - ADC. Add IP (+)
and search for *AD1*. Select *AD1Pmod* and not the *PmodAD1_v1_0* from the Digilent Pmod library.



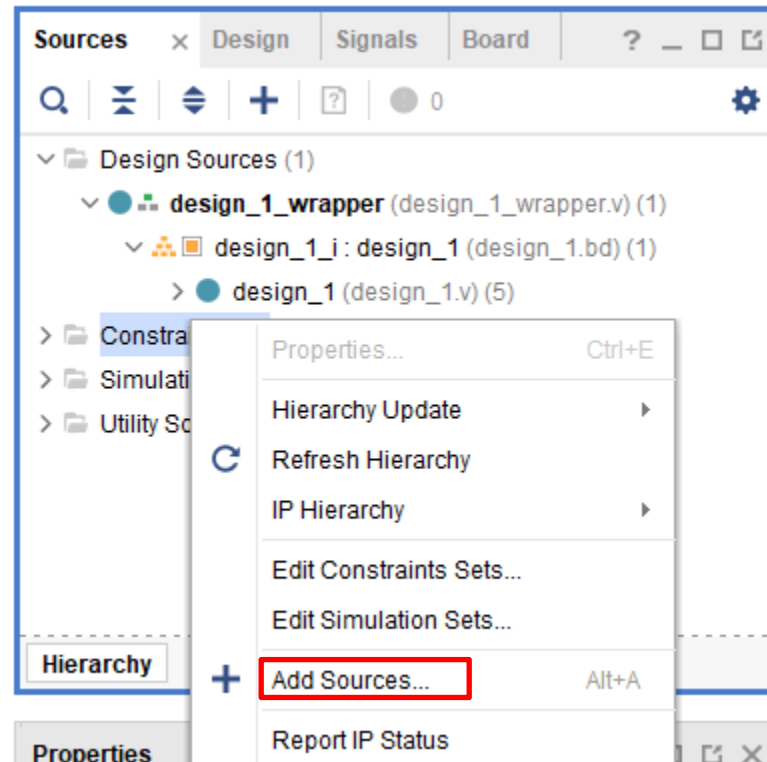
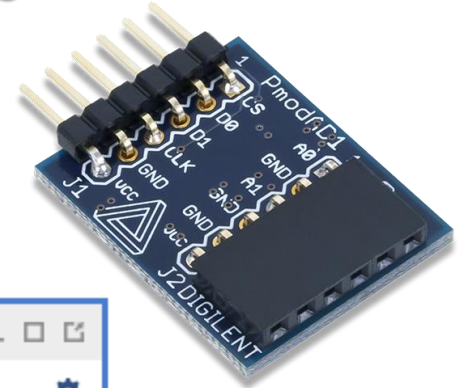
Zynq SPI Peripheral - ADC. The *Pmod_AD1_v1_0* is configured by Digilent and uses a GPIO interface which result in a relatively poor throughput rate R_{ADC} .



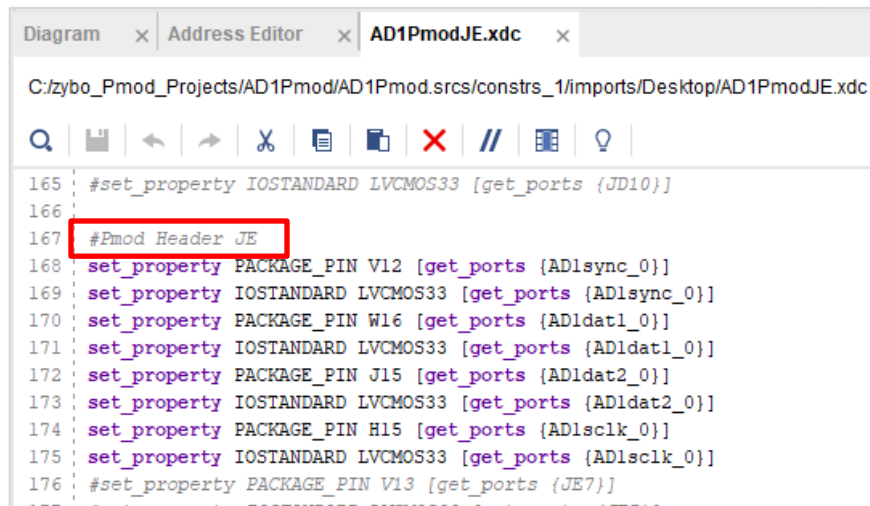
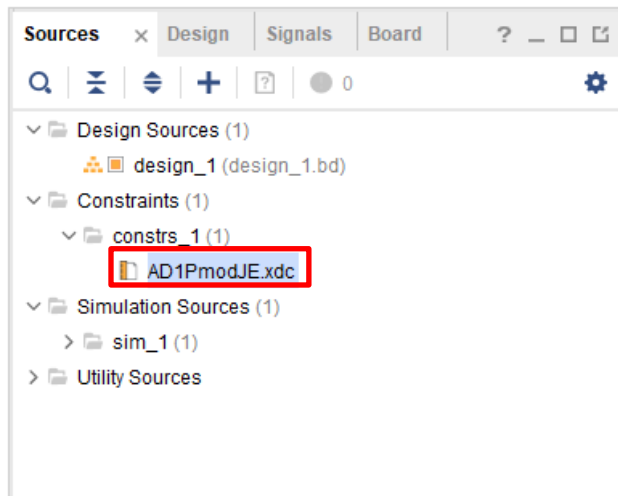
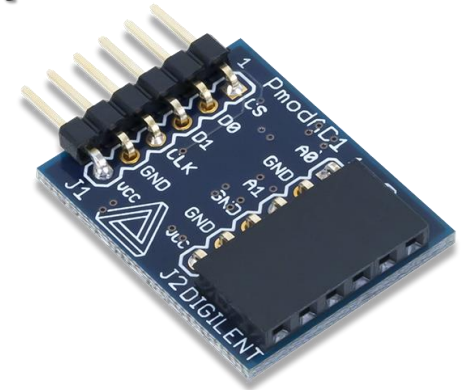
Zynq SPI Peripheral - ADC. Run Connection Automation to add the Reset and AXI interconnect.



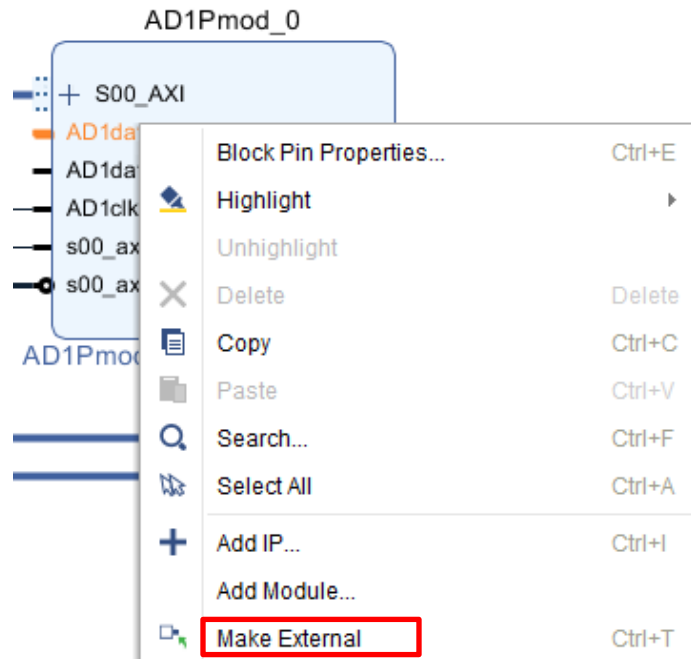
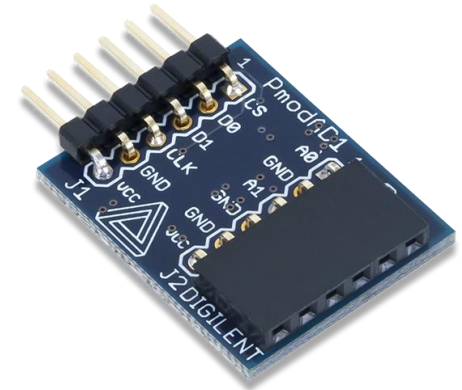
Zynq SPI Peripheral - ADC. In the *Sources* tab under *Constraints*, right-click and *Add Sources*. The constraint file is *AD1PmodJE.xdc* on *Canvas*.



Zynq SPI Peripheral - ADC. The constraint file *AD1PmodJE.xdc* has the Pmod AD1 named connections to the top row of port JE (left most, bottom).



Zynq SPI Peripheral - ADC. Right-click on each of the four unconnected pins of the AD1 (*AD1dat1*, *AD1dat2*, *AD1sync*, and *AD1sclk*) and select *Make External*. The named connection is routed to port JE by the constraint file *AD1PmodJE.xdc*.



Zynq SPI Peripheral - ADC. The AD1 Pmod uses an external 30 MHz clock signal. Right-click on the Zynq PS and select *Customize Block*, then select *Clock Configuration*.

The image shows the Vivado IDE interface. On the left, a Zynq PS block is highlighted in the block design, with a context menu open. The menu options include Block Properties..., Highlight, Unhighlight, Delete, Copy, Paste, Search..., Select All, Add IP..., Add Module..., Make External, Customize Block..., IP Documentation, and Orientation. The 'Customize Block...' option is highlighted with a red rectangle. On the right, the 'Zynq Block Design' window is open, showing the 'ZYNQ7 Processing System (5.5)' configuration. The 'Page Navigator' on the left lists various configuration sections, with 'Clock Configuration' highlighted with a red rectangle. The main area of the window displays a detailed block diagram of the Zynq7 Processing System, including components like I/O MUX, Bank0 MIO, Bank1 MIO, I/O Peripherals, General Settings, Application Processor Unit (APU), ARM Cortex-A9 CPU, Snoop Control Unit, 512 KB L2 Cache and Controller, OCM Interconnect, 256 KB SRAM, Memory Interfaces, DDR2/3L PDDR2 Controller, Processing System (PS), and Programmable Logic (PL). The diagram shows the interconnections between these components, including the central interconnect, DMA channels, and various clocks and resets.

Zynq SPI Peripheral - ADC. Expand *PL Fabric Clocks* and select *FCLK_CLK1* and set to 30 MHz. OK to continue.

Re-customize IP

ZYNQ7 Processing System (5.5)

Documentation Presets IP Location Import XPS Settings

Page Navigator

- Zynq Block Design
- PS-PL Configuration
- Peripheral I/O Pins
- MIO Configuration
- Clock Configuration**
- DDR Configuration
- SMC Timing Calculation
- Interrupts

Clock Configuration [Summary Report](#)

Basic Clocking **Advanced Clocking**

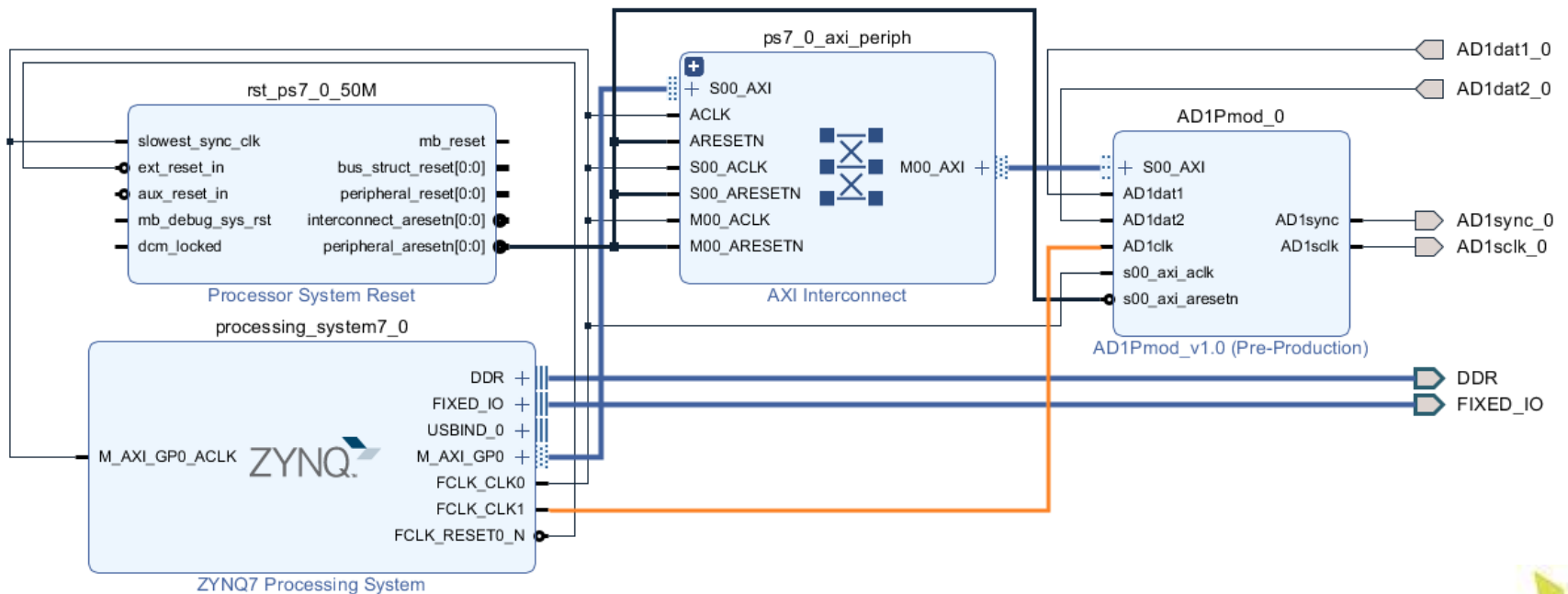
Input Frequency (MHz) 50 CPU Clock Ratio 6:2:1

Search: Q

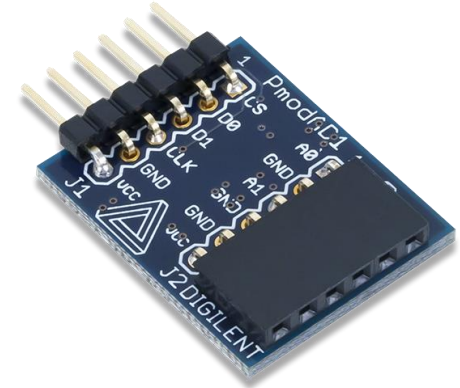
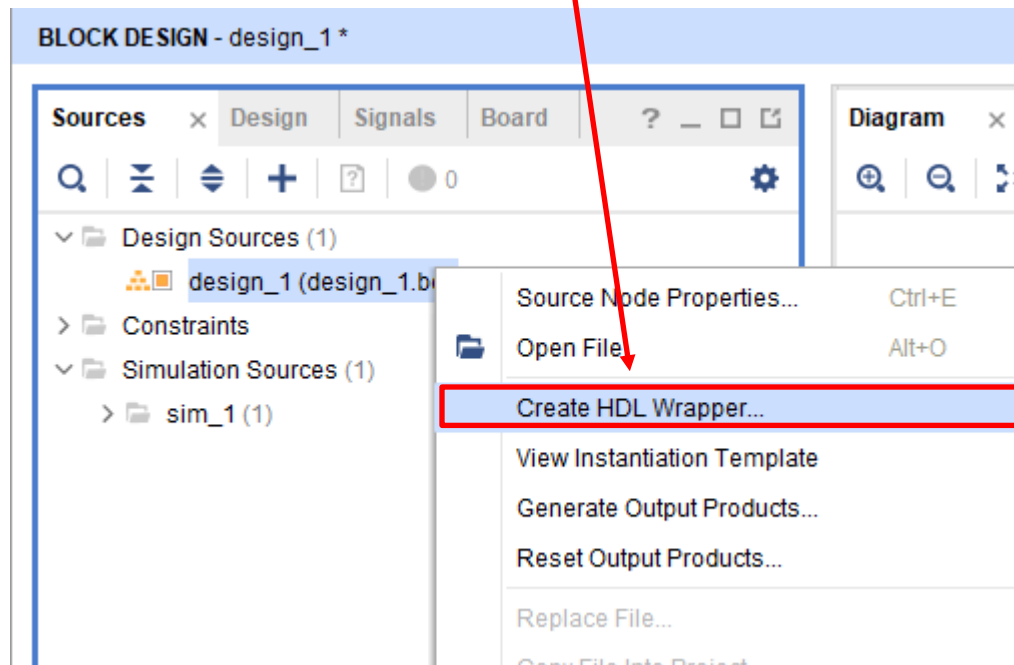
Component	Clock Source	Requested Frequ...	Actual Frequency(...)	Range(MHz)
> Processor/Memory Clocks				
> IO Peripheral Clocks				
▼ PL Fabric Clocks				
☒ FCLK_CLK0	IO PLL	100	100.000000	0.100000 : 250.000000
☒ FCLK_CLK1	IO PLL	30	30.303030	0.100000 : 250.000000
☐ FCLK_CLK2	IO PLL	50	10.000000	0.100000 : 250.000000
☐ FCLK_CLK3	IO PLL	50	10.000000	0.100000 : 250.000000
> System Debug Clocks				
> Timers				

OK Cancel

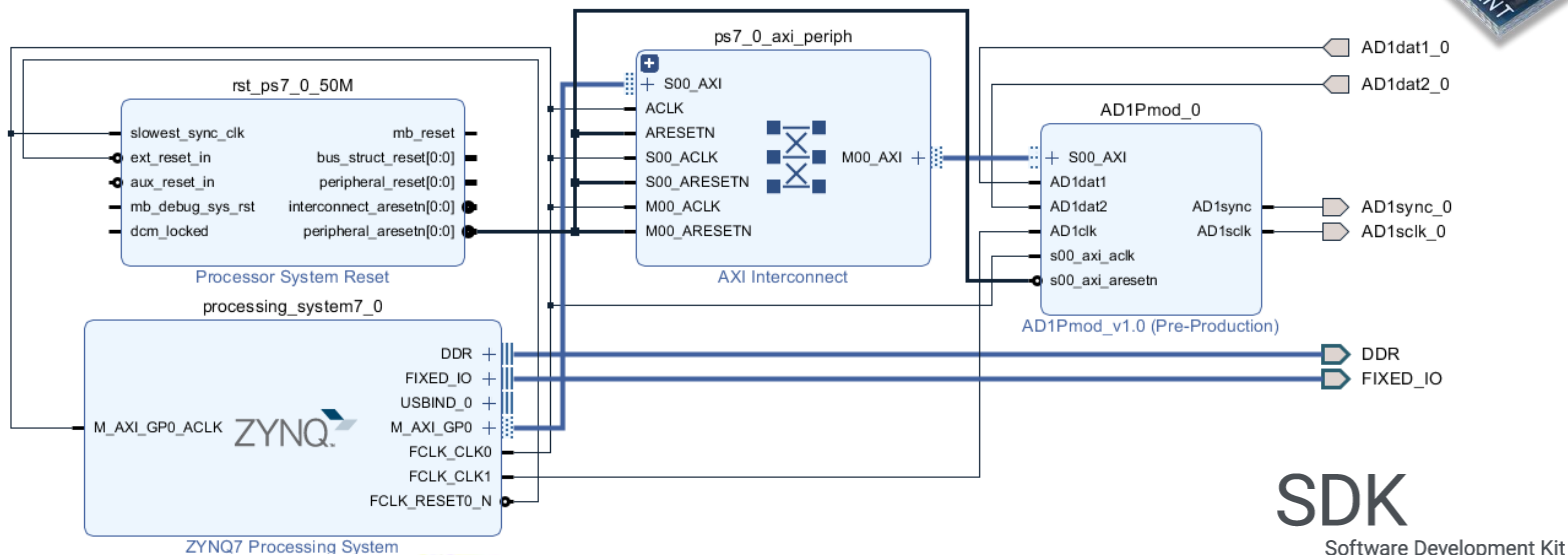
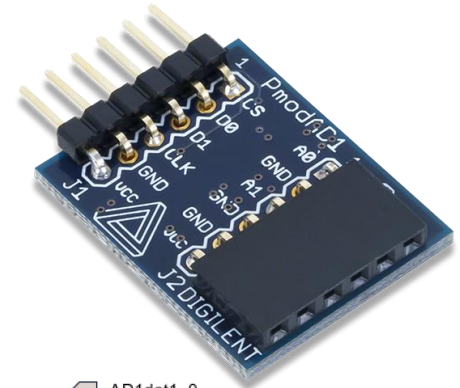
Zynq SPI Peripheral - ADC. The *FCLK_CLK1* port of the Zynq PS is manually connect to the *AD1clk* port of the AD1Pmod IP.



Zynq SPI Peripheral - ADC. In the *Block Design...Sources* tab right-click *design_1* and *Create HDL Wrapper*.



Zynq SPI Peripheral - ADC. In the *Flow Navigator...Generate Bitstream* for the Hardware design. Select *File...Export...Hardware*. Remember to include bitstream ✓ then *File...Launch SDK*.



SDK
Software Development Kit

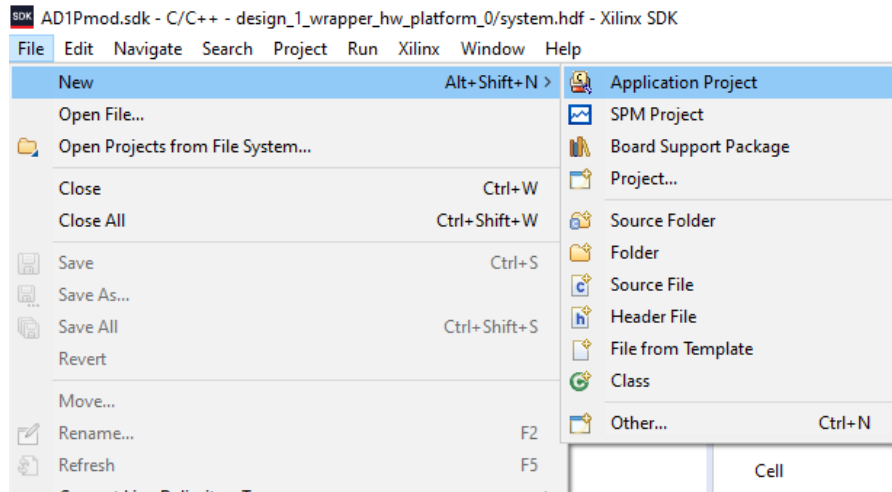
VIVADOTM

2018.2

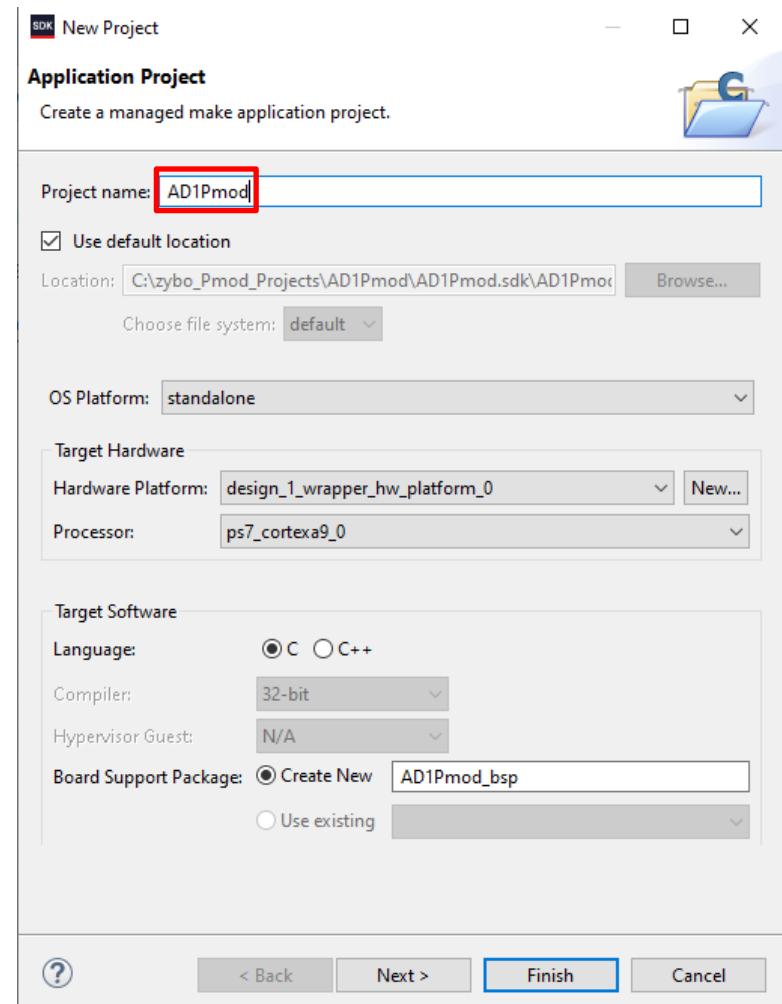
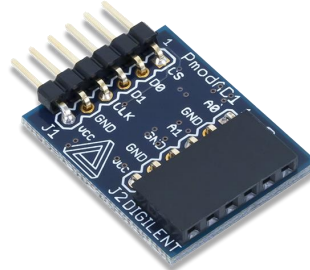
XILINX.

Copyright 1986-2018 Xilinx, Inc.
All Rights Reserved.

Zynq SPI Peripheral - ADC. In SDK open *File...New...Application Project*

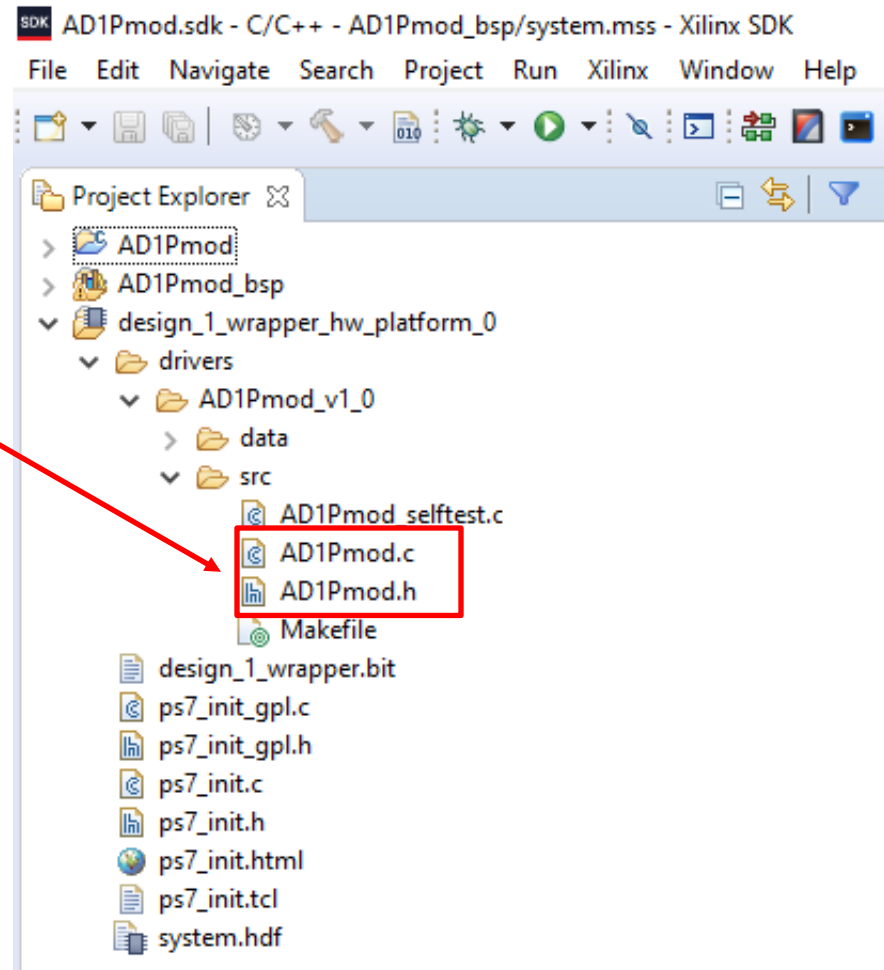
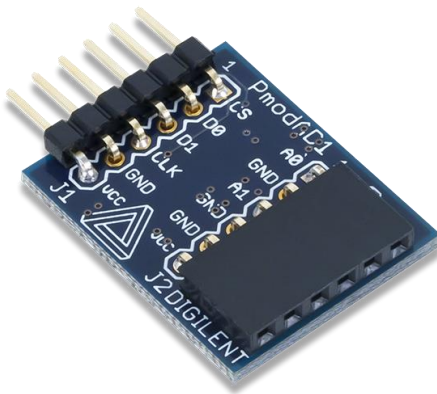


Project name is *AD1Pmod* as a *standalone* and all other options as provided.



Zynq SPI Peripheral - ADC. In *SDK...Project Explorer...Drivers* the AD1Pmod software should appear.

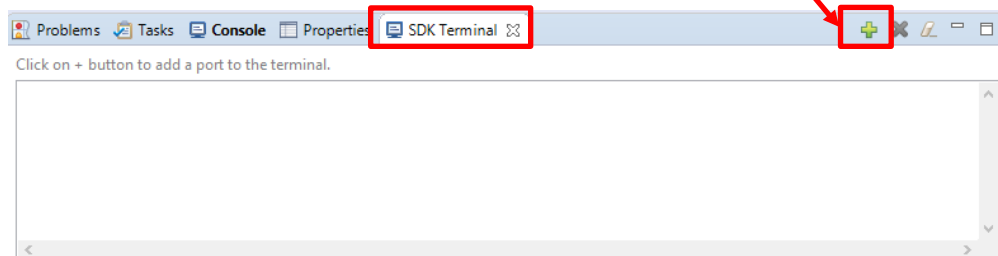
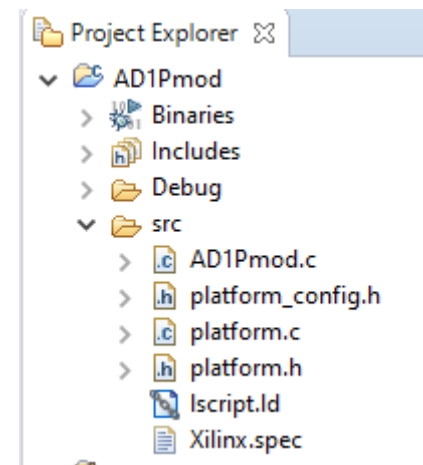
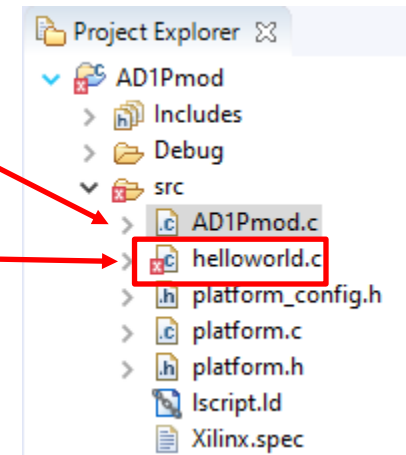
Under src (source) *AD1Pmod.c* is the project stub file and the *AD1Pmod.h* include file for definitions.



Zynq SPI Peripheral - ADC. Download *AD1Pmod.c* from *Canvas* and import it into *AD1Pmod...src*

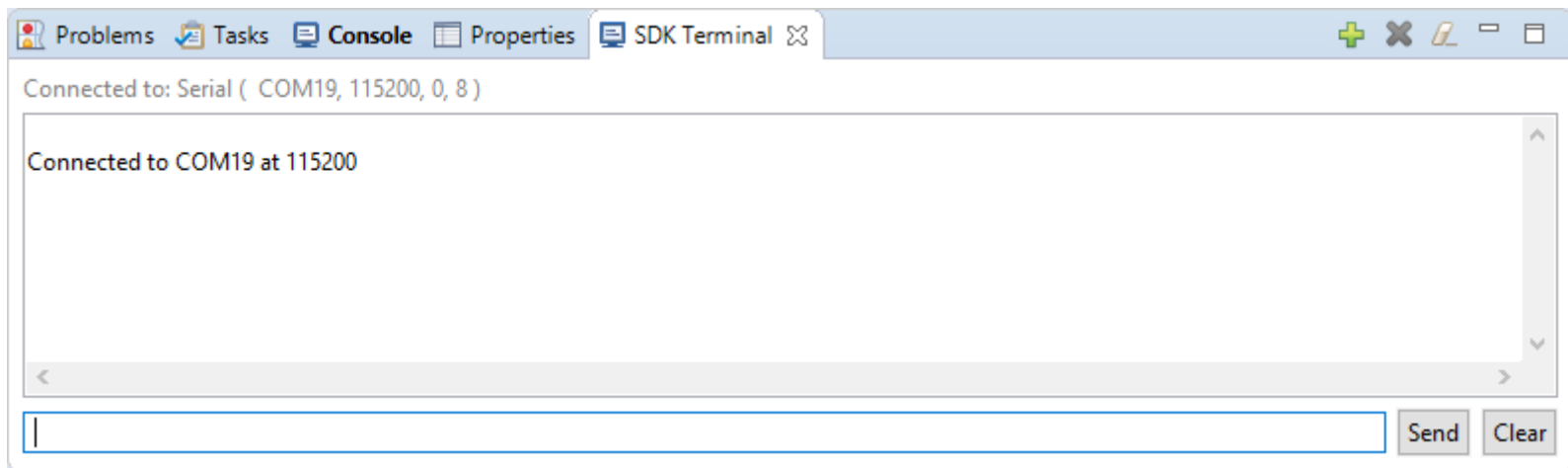
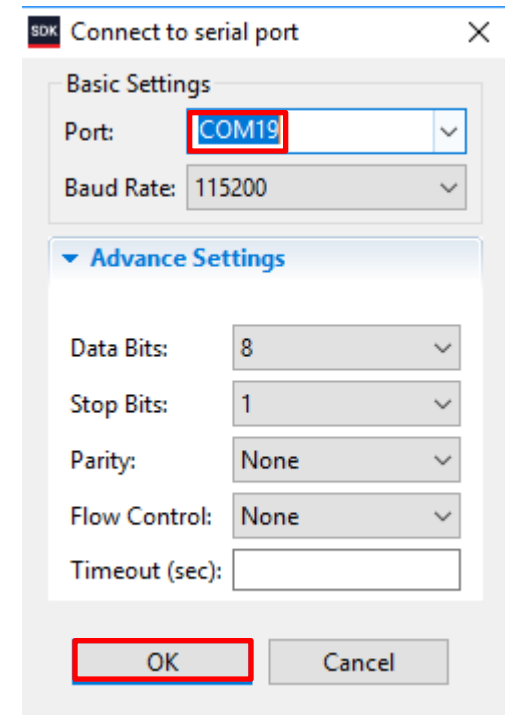
The *helloworld.c* program is deleted from this folder.

The program output appears in the SDK Terminal. Click **+** to open a terminal port.



Zynq SPI Peripheral – ADC. The Port (may be) automatically identified. This is the same USB port used for programming the Zybo and providing power.

The default baud (bit) rate, data and stop bits, parity and flow control are used. *OK* to continue.



Zynq SPI Peripheral - ADC. The AD1Pmod base addresses are in the Address Editor in Vivado and are 32-bit registers (4 bytes).

```
//AD1Pmod.c PmodAD1 ADC example ECE3622
```

```
#include "xparameters.h"
#include "xil_io.h"
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include <sleep.h>
```

```
//AD1Pmod from Address Editor in Vivado
```

```
#define AD1acq 0x43C00000 //AD1 acquisition
#define AD1dav 0x43C00004 //AD1 data available
#define AD1dat1 0x43C00008 //AD1 channel 1 data
#define AD1dat2 0x43C0000C //AD1 channel 2 data
```

```
int main(void)
```

```
{
    int adcdav;           //ADC data available
    int adcdat1;          //ADC channel 1 data
    int adcdat2;          //ADC channel 2 data
```

```
    xil_printf("\n\nStarting AD1 Pmod demo test...\n");
    Xil_Out32(AD1acq,0);
```

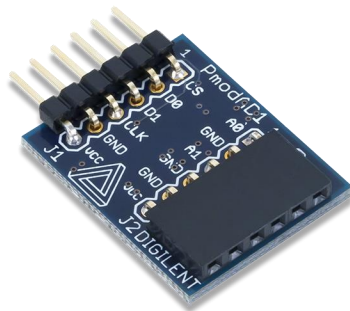
```
    while (1)
```

```
    {
        Xil_Out32(AD1acq,0);           //ADC stop acquire
        adcdav=Xil_In32(AD1dav);       //ADC available?
```

Diagram x Address Editor x					
Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
AD1Pmod_0	S00_AXI	S00_AXI_reg	0x43C0_0000	64K	0x43C0_FFFF

Zynq SPI Peripheral – ADC. *AD1Pmod.c* with hardware *handshaking*: AD1acq start the ADC data acquisition and AD1dav data available signals when completed.

The unipolar ADC values are outputted to the SDK terminal.



```
//AD1Pmod.c PmodAD1 ADC example ECE3622 c2019 Dennis Silage

#include "xparameters.h"
#include "xil_io.h"
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include <sleep.h>

//AD1Pmod from Address Editor in Vivado
#define AD1acq 0x43C00000 //AD1 acquisition
#define AD1dav 0x43C00004 //AD1 data available
#define AD1dat1 0x43C00008 //AD1 channel 1 data
#define AD1dat2 0x43C0000C //AD1 channel 2 data

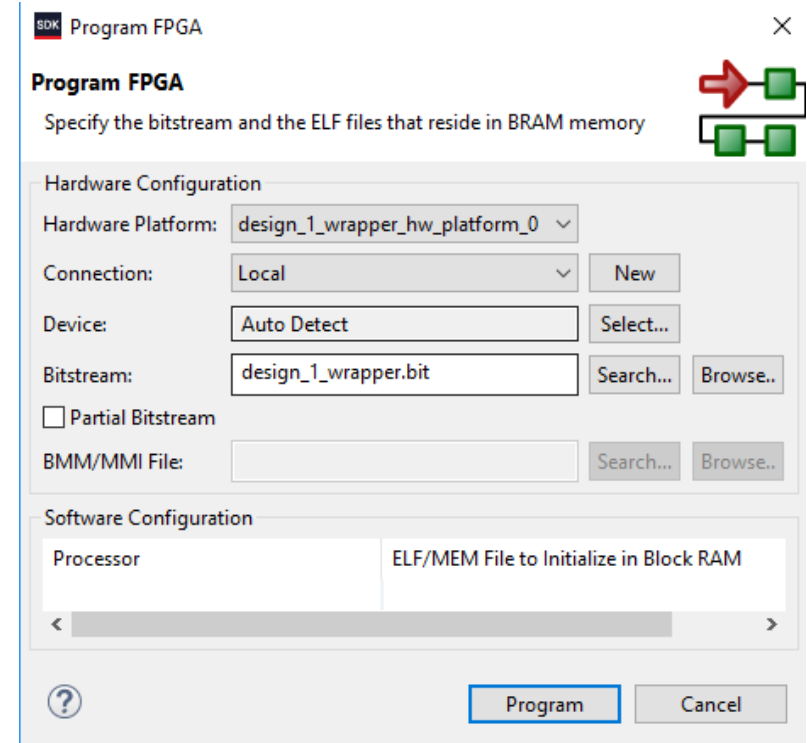
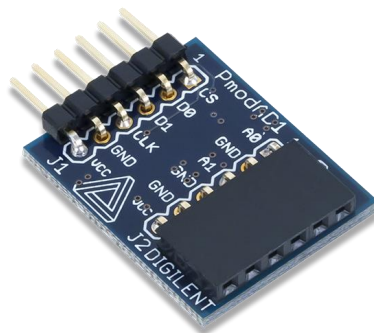
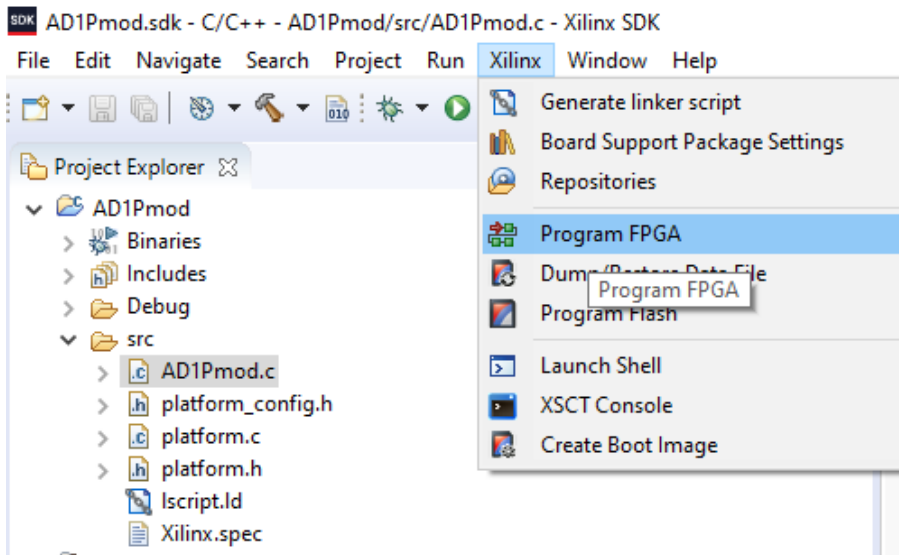
int main(void)
{
    int adcdav;           //ADC data available
    int adcdat1;          //ADC channel 1 data
    int adcdat2;          //ADC channel 2 data

    xil_printf("\n\rStarting AD1 Pmod demo test...\n");
    Xil_Out32(AD1acq,0);

    while (1)
    {
        Xil_Out32(AD1acq,0);           //ADC stop acquire
        adcdav=Xil_In32(AD1dav);        //ADC available?
        while(adcdav==1)
            adcdav=Xil_In32(AD1dav);

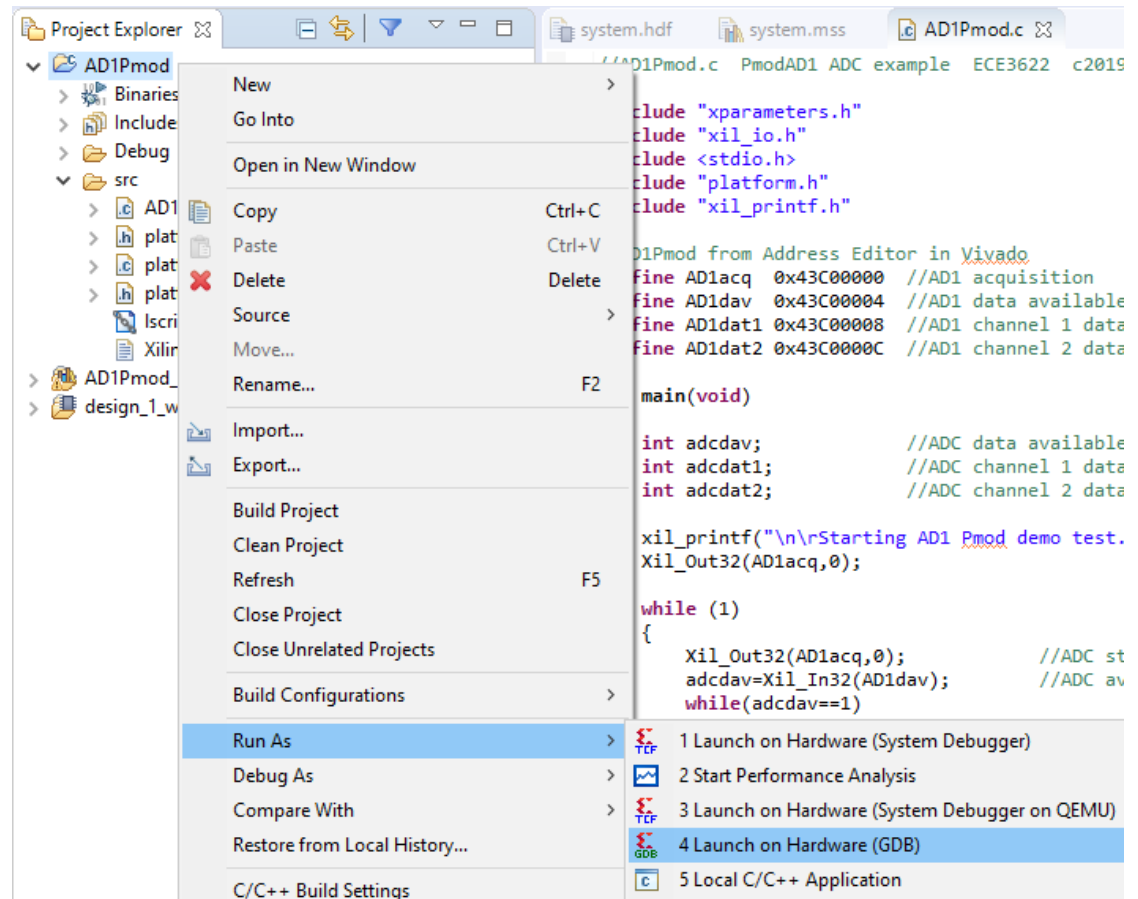
        while(1)
        {
            Xil_Out32(AD1acq,1);       //ADC acquire
            while(adcdav==0)
                adcdav=Xil_In32(AD1dav); //ADC data?
            Xil_Out32(AD1acq,0);        //ADC stop acquire
            adcdat1=Xil_In32(AD1dat1);
            adcdat2=Xil_In32(AD1dat2);
            while(adcdav==1)
                adcdav=Xil_In32(AD1dav); //ADC reset?
            xil_printf("%d Ch1 %d, Ch2 %d\n", adcdat1, adcdat2);
            sleep(1);
        }
    }
}
```

Zynq SPI Peripheral - ADC. In *SDK* on the *Xilinx* tab select *Program FPGA*.

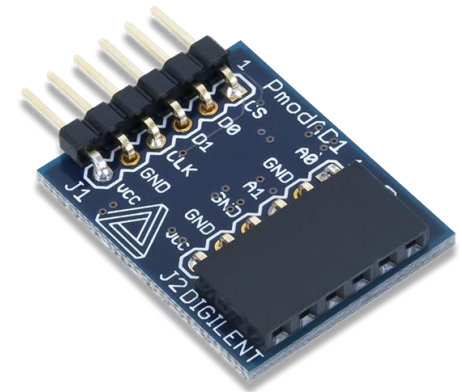
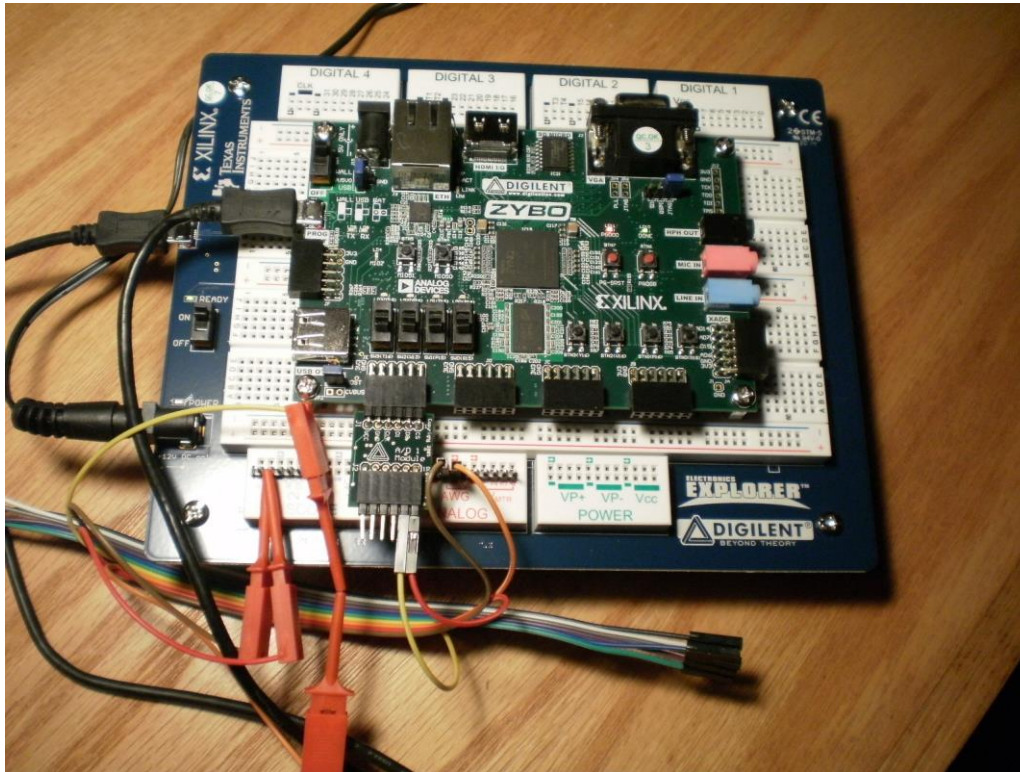


Zynq SPI Peripheral - ADC. In *SDK...Project Explorer* under the project name *AD1Pmod* right-click and select *Run As...Launch on Hardware (GDB)*.

Program output appears in the SDK terminal. Source code changes when saved and re-built before execution.

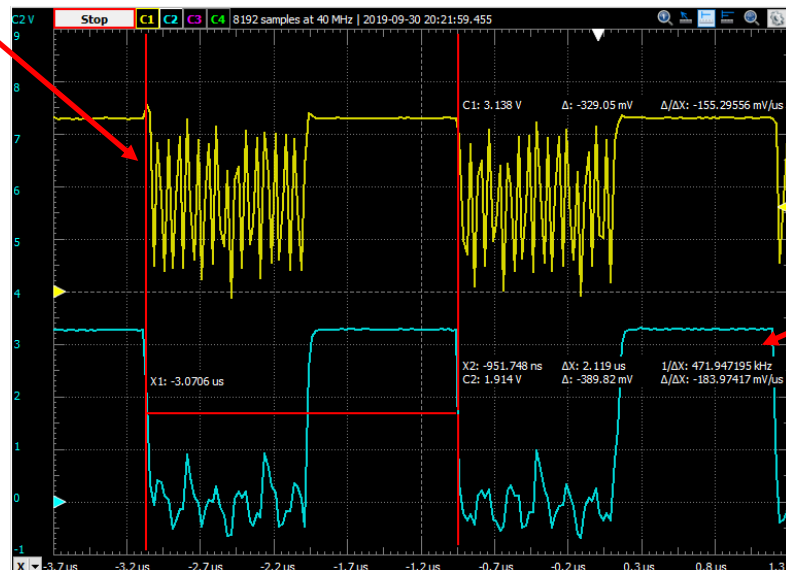
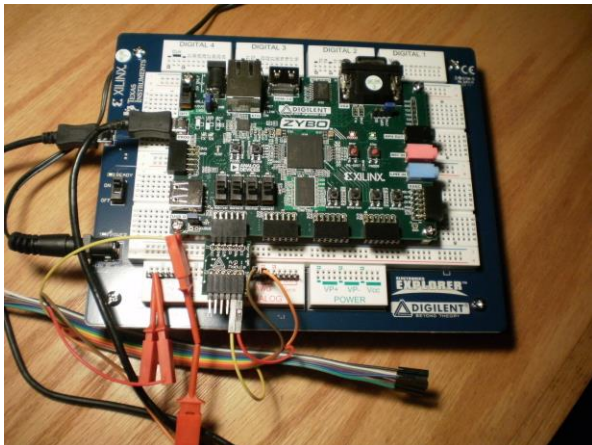
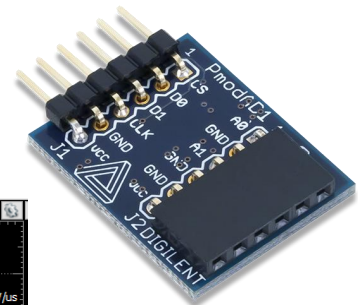


Zynq SPI Peripheral - ADC. Here the *EEBoard* is used to provide a DC or slowly varying sinusoidal signal to the AD1Pmod and to measure the ADC data rate R_{ADC} in samples per second.



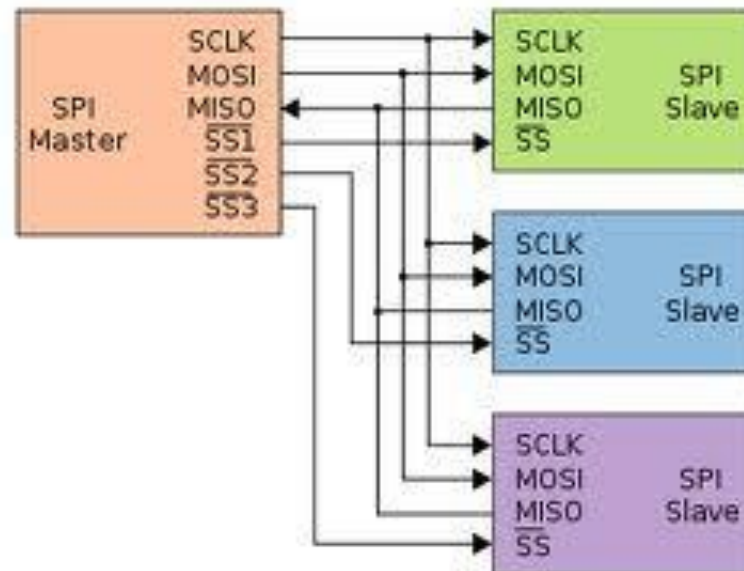
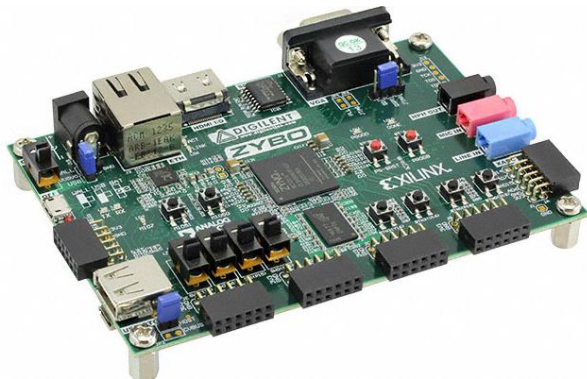
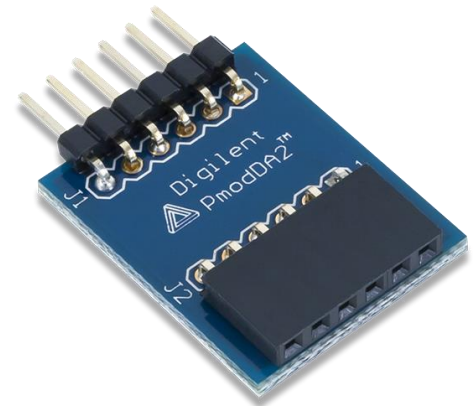
Zynq SPI Peripheral - ADC. For the AD1Pmod template project after removing the printout of values and sleep(1), the measured CS (chip select or SS) pulse is active low for 1.17 μ sec and inactive high for 0.96 μ sec. The active duty cycle for ADC conversion is $1.17/(1.17+0.96) \approx 54.9\%$ and R_{ADC} is $1/(1.17+0.96 \mu\text{sec}) \approx 496$ ksamples/sec

SCLK

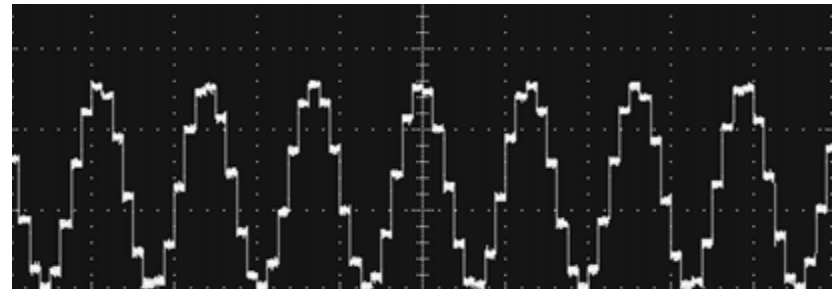
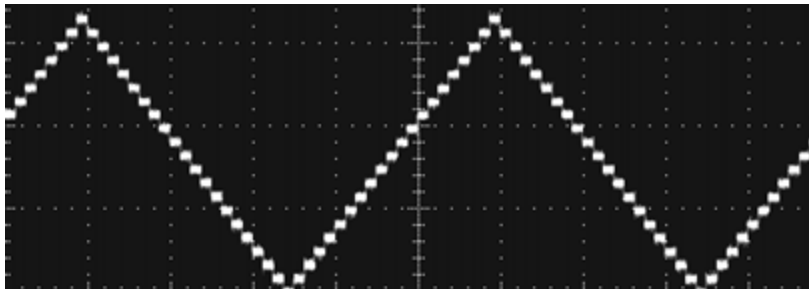
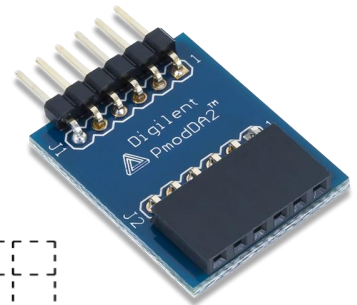
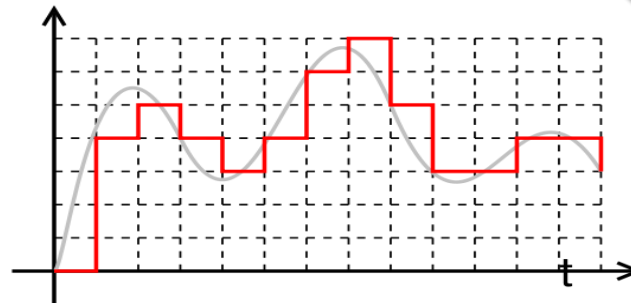
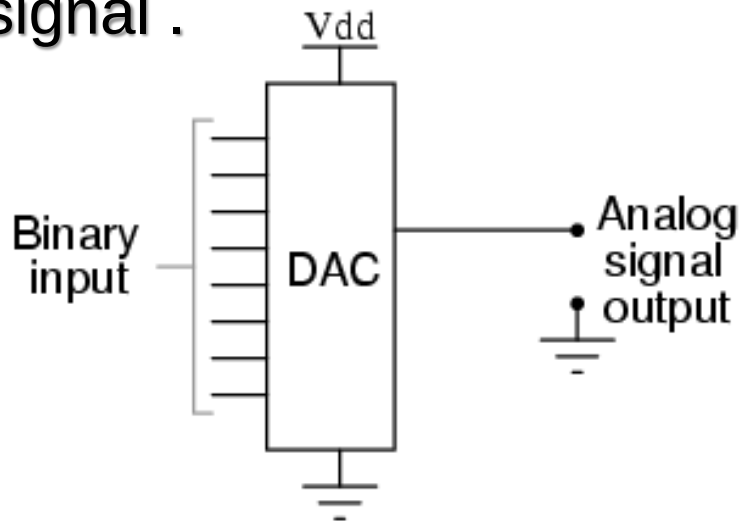


SS

Zynq SPI Peripheral - DAC. As another example of SPI bus interfacing to a Zynq device, a Digilent Pmod DA2 two-channel, 12-bit resolution digital-to-analog converter (DAC) is interfaced to the Zynq PS in the Vivado HLx design project using the extended *ZynqPmodVivado-Library*.



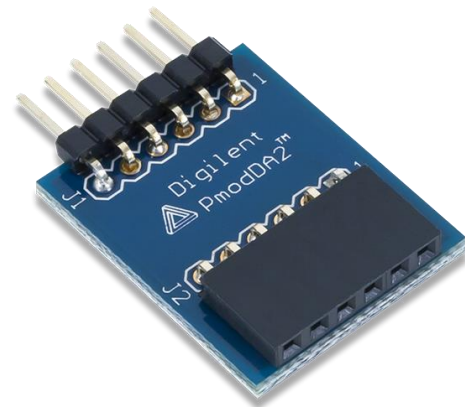
Zynq SPI Peripheral - DAC. The DAC provides an analog output signal for the discrete sampled data of an application as a *step-wise* but continuous analog output signal .



Zynq SPI Peripheral - DAC. The DAC provides an analog output signal for embedded system design in audio processing, analog and digital baseband and bandpass communication and digital process control.

The analog output voltage resolution (or step size) ΔV for a DAC that inputs an unsigned n-bit binary values is determined by:

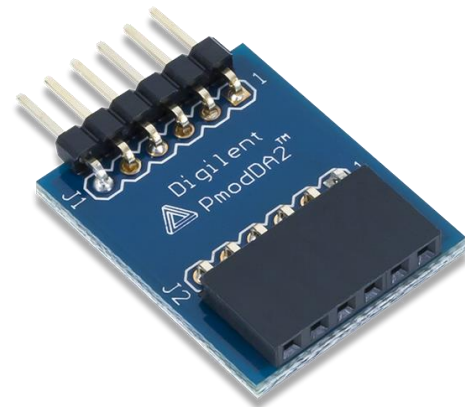
$$\Delta V = \frac{V_{\text{REF}}}{2^n}$$



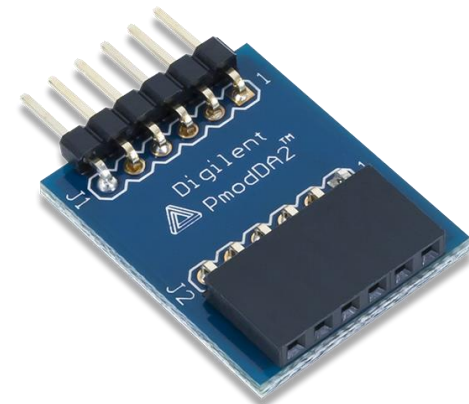
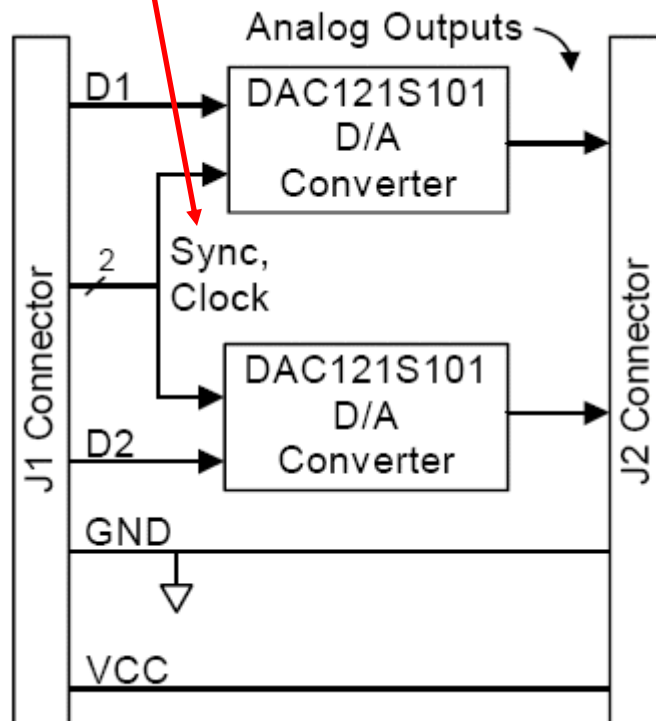
Zynq SPI Peripheral - DAC. V_{REF} is the precision DC voltage used as a reference. The maximum rate at which the DAC can accept binary data and update its analog voltage output is its throughput rate R_{DAC} in samples per second.

The Digilent PmodDA2 HAS two channels, each with 12-bits of resolution.

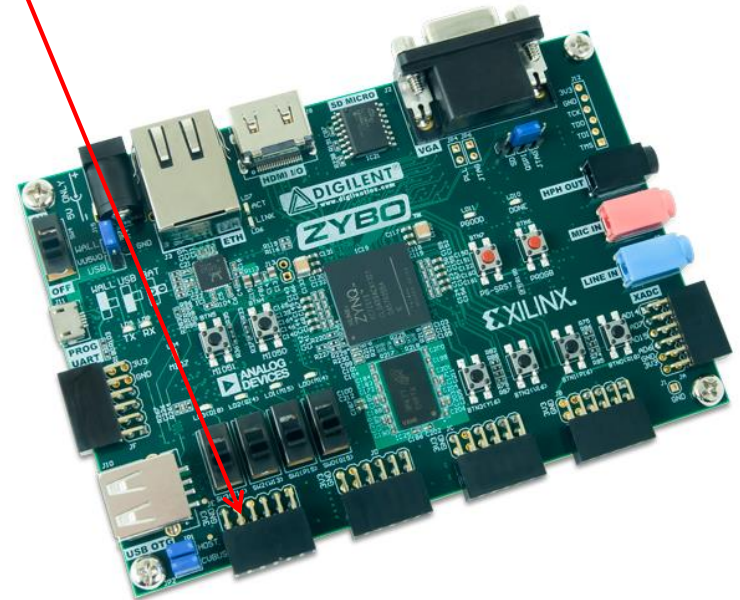
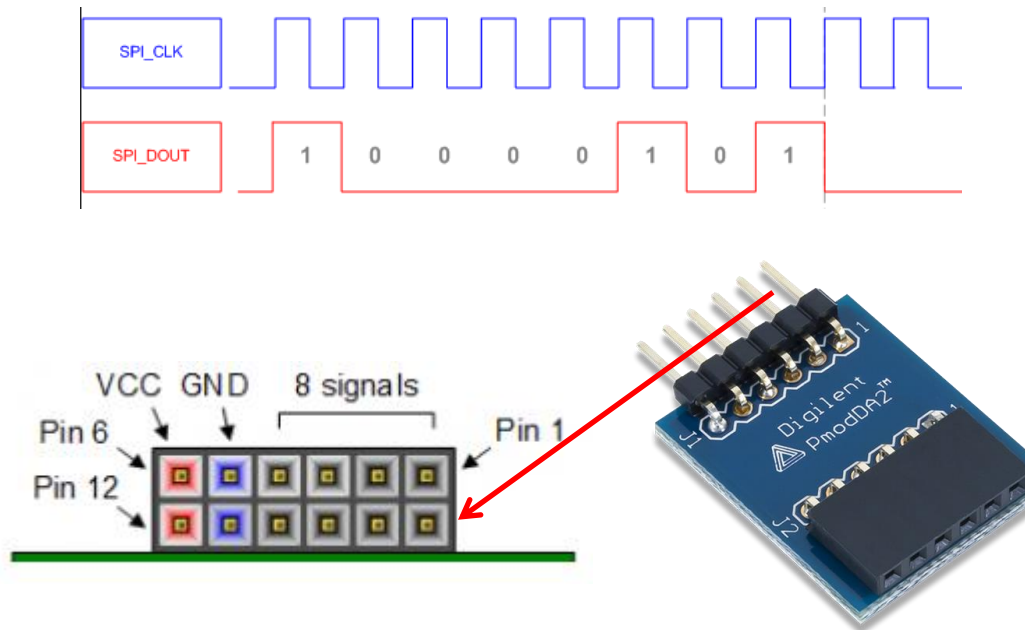
$$\Delta V = \frac{V_{REF}}{2^n}$$



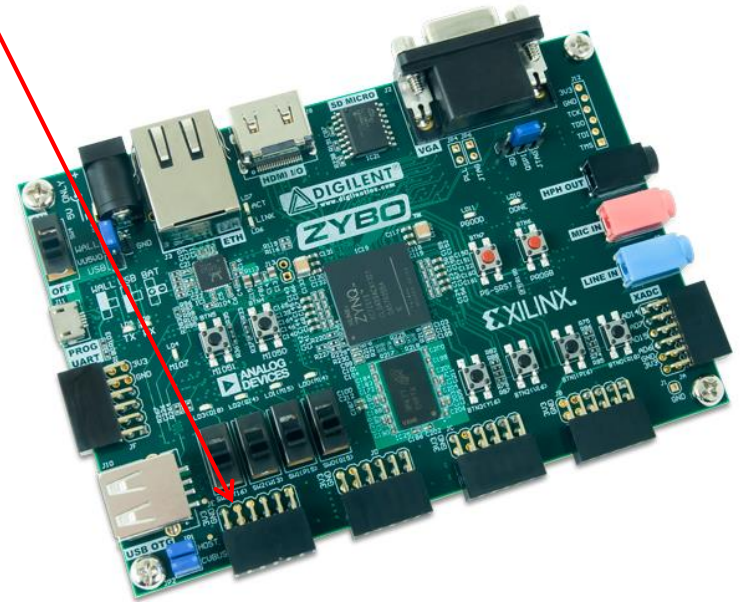
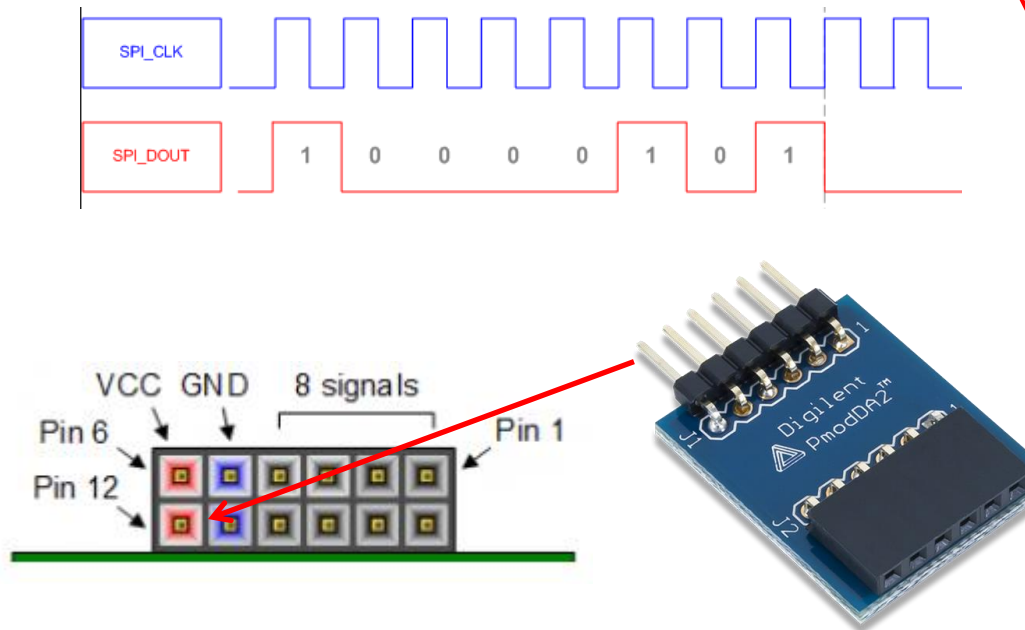
Zynq SPI Peripheral - DAC. The PmodDA2 hardware module features two Texas Instruments (www.ti.com) DAC121S101 12-bit DACs and utilizes the SPI protocol with a 16-bit data packet (12 bits of data and 4 bits of control). SYNC is the slave select signal SS.



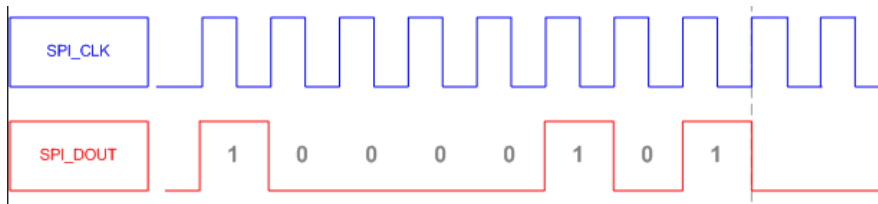
Zynq SPI Peripheral - DAC. The PmodDA2 DAC is connected to the *bottom 6 pins* of the 12-pin peripheral hardware module connector (je) of the Zybo Board. .



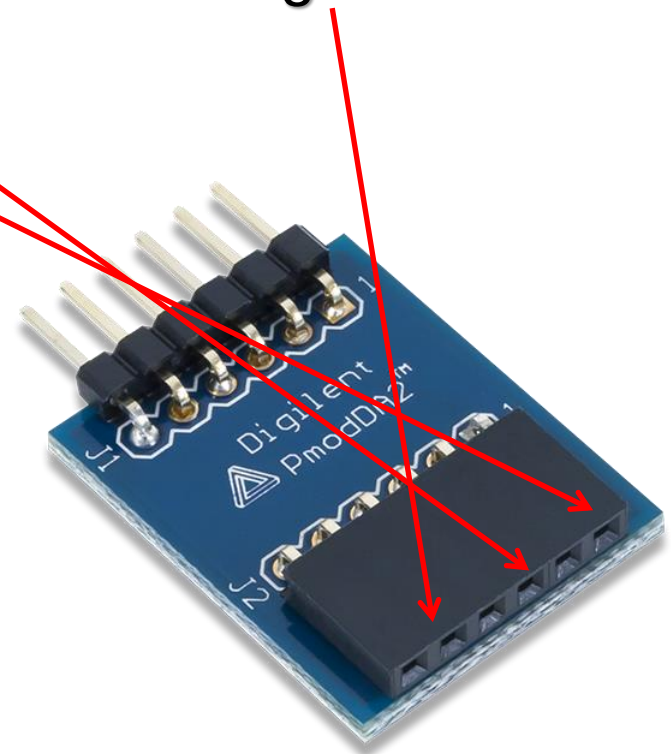
Zynq SPI Peripheral - DAC. The SYNC (or SS) signal is pin je[7], the two input data signals DACD0 and DACD1 are pins je[8] and je[9] and the SPI bus clock signal SCLK is pin je[10]. Pins je[11] and je[12] are DC ground and power ($VDD = +3.3\text{ V DC}$).



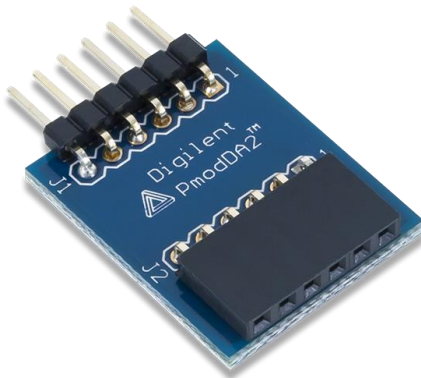
Zynq SPI Peripheral - DAC. The analog output voltage for the 12-bit DAC is determined by the equation below with $n = 12$, $D[11:0]$ ranging from 0 to 4095 and $V_{REF} = 3.3$ V. The two unipolar analog output voltages A and B appear on a 6-pin header at pins 1 and 3 and ground at pin 5.



$$V_{OUT} = \frac{D[n-1:0] \times V_{REF}}{2^n} \text{ V}$$



Digital to Analog Converter. Installation and use of the DAC is also described in the *PmodAD1* and *PmodDA2* Peripherals on Canvas.

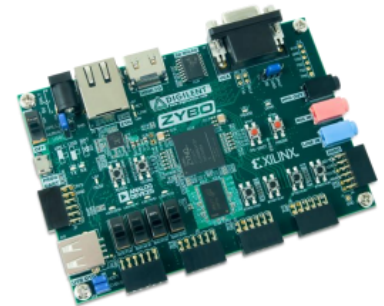


ECE3623 Embedded System Design Laboratory

Dennis Silage, PhD
silage@temple.edu

PmodAD1 and PmodDA2 Peripherals

The Digilent Zybo Board does not feature an analog-to-digital converter (ADC) or digital-to-analog converter (DAC). The Digilent PmodAD1 and PmodDA2 provide this functionality and both 6-pin peripheral devices can be installed on the 12-pin Pmod jack on the lower left side of the Zybo Board.



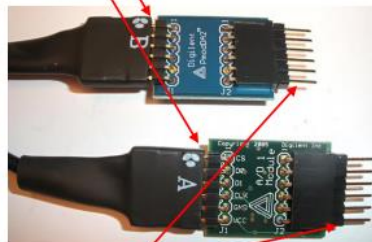
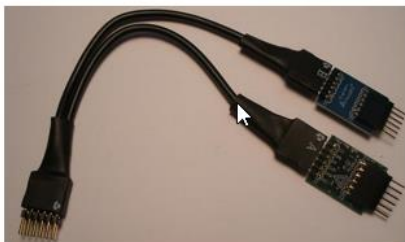
The leftmost, front Pmod jack (J1) is available from the programmable logic (PL) of the Zynq SoC device and is shown as a *front view*. Note the location of pin 1 on the top right. Pin 7 is immediately below pin 1 on the bottom right. To facilitate the connection a 12 pin to two 6 pin cable is provided as shown in the Figure.



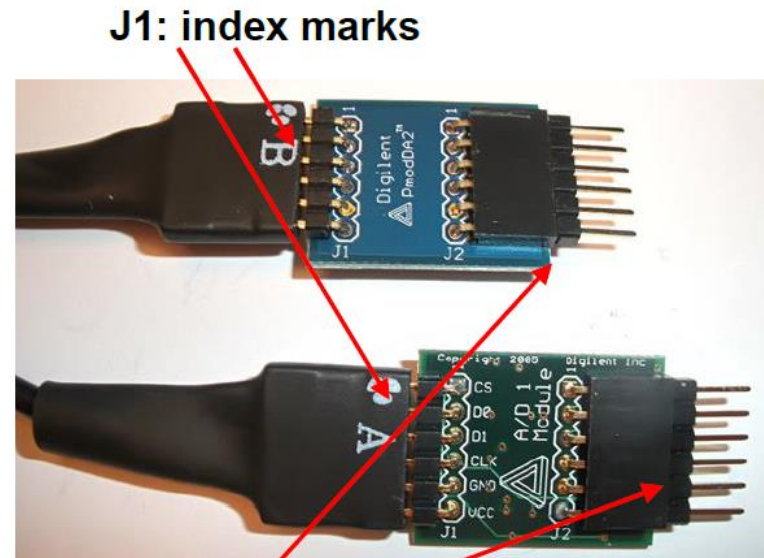
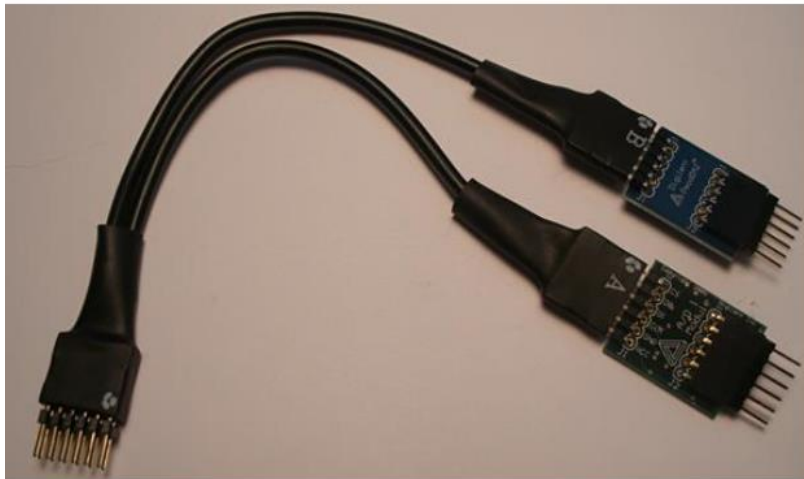
The PmodAD1 ADC is connected to the cable 6 pin connector A with the index mark positioned on pin 1 of J1 the SPI interface, as shown in the Pictures. The PmodDA2 is connected to the cable 6 pin connector B with the index mark position on pin 1 of J1 the SPI interface as shown in the Pictures.

J1: index marks

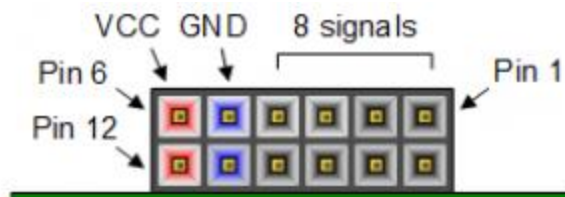
J2: 6 pin headers



Zynq SPI Peripheral - DAC. Installation and use of the DAC is described in the *PmodAD1* and *PmodDA2* Peripherals on Canvas.



J2: 6 pin headers input or output connectors

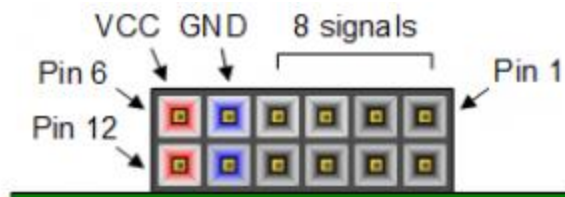


Zynq SPI Peripheral - DAC. Installation and use of the ADC is described in the *PmodAD1* and *PmodDA2* Peripherals on Canvas.

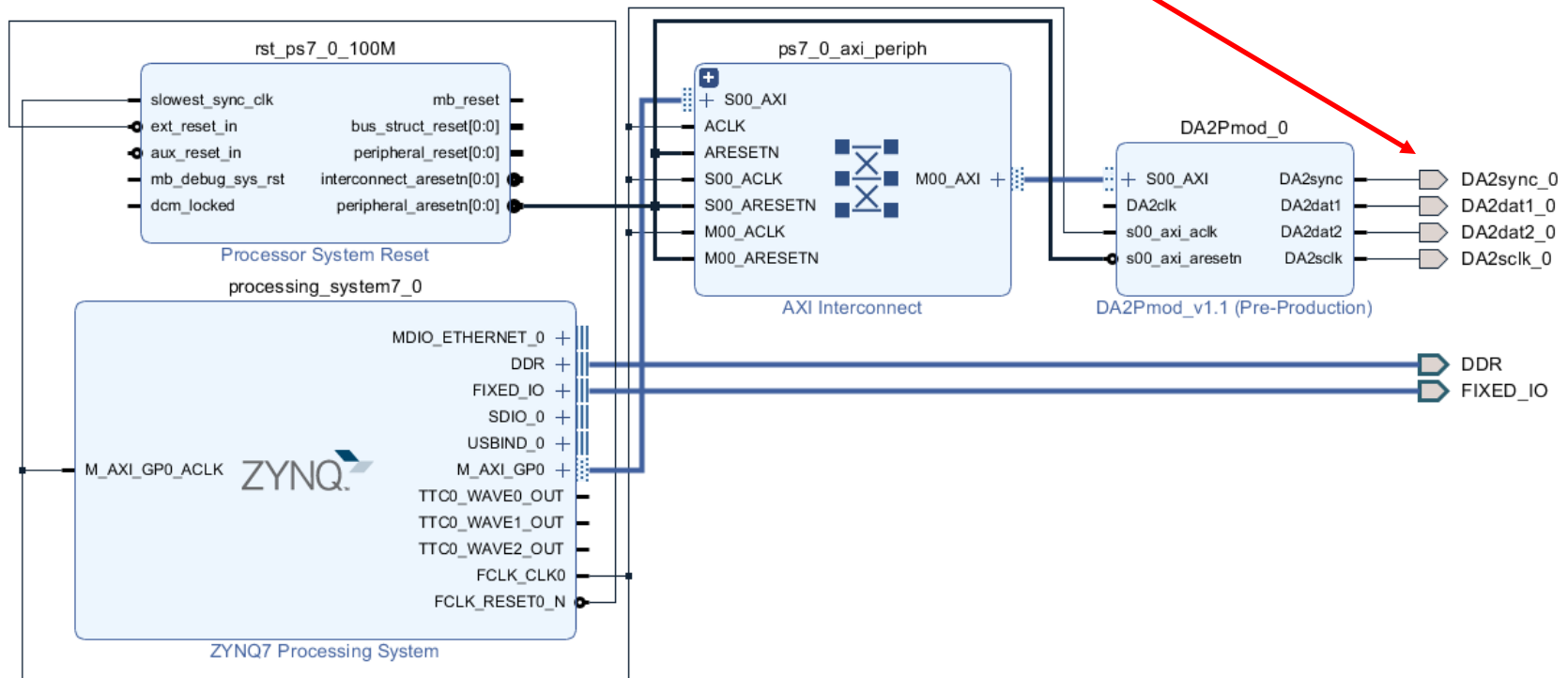
Zybo Board Pmod Jack

The 12 pin connector of the cable is provided with a 12 pin header which is inserted into the 12-pin Pmod jack on the lower left side of the Zybo Board (JE). The index mark shown in the picture **must be on the lower right side and not visible from the top surface** of the Zybo Board.

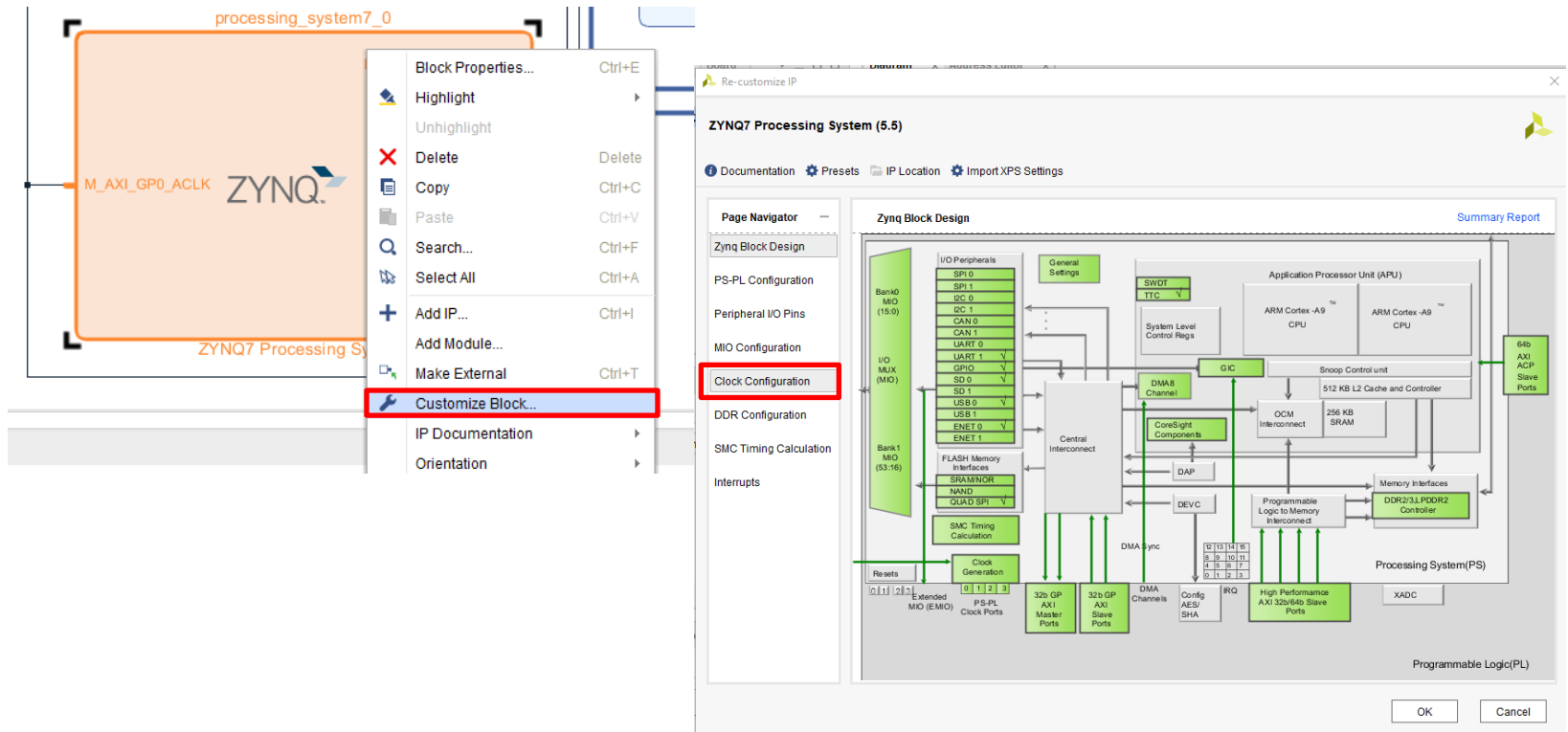
This insures that V_{CC} for the PmodAD1 and PmodDA2 are connected properly at pins 6 and 12 of the 12 pin Pmod connector of the Zybo Board. **Improper connection can damage the Zybo Board and the Pmod peripherals and is your responsibility.**



Zynq SPI Peripheral - DAC. Create a Vivado block design and add the *DA2Pmod* IP. As for the *AD1Pmod* IP make the four *DA2Pmod* outputs pins external.



Zynq SPI Peripheral - DAC. The DA2Pmod uses an external 50 MHz clock signal. Right-click on the Zynq PS and select *Customize Block*, then select *Clock Configuration*.



Zynq SPI Peripheral - DAC. Expand *PL Fabric Clocks* and select *FCLK_CLK1* and set to 50 MHz. OK to continue.

Re-customize IP

ZYNQ7 Processing System (5.5)

Documentation Presets IP Location Import XPS Settings

Page Navigator

- Zynq Block Design
- PS-PL Configuration
- Peripheral I/O Pins
- MIO Configuration
- Clock Configuration
- DDR Configuration
- SMC Timing Calculation
- Interrupts

Clock Configuration

Basic Clocking Advanced Clocking

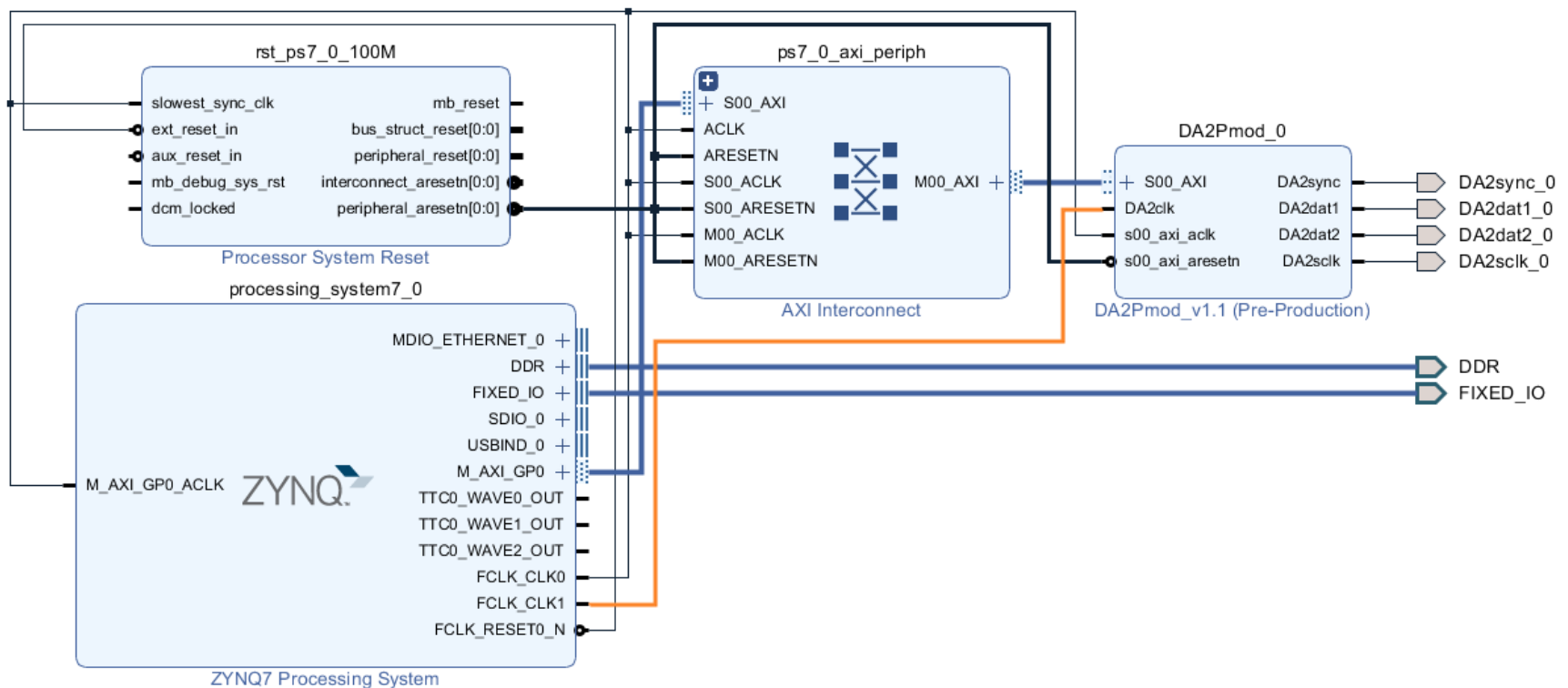
Input Frequency (MHz) 50.000000 CPU Clock Ratio 6:2:1

Search: Q:

Component	Clock Source	Requested Frequ...	Actual Frequency(...)	Range(MHz)
> Processor/Memory Clocks				
> IO Peripheral Clocks				
> PL Fabric Clocks				
☒ FCLK_CLK0	IO PLL	100	100.000000	0.100000 : 250.000000
☒ FCLK_CLK1	IO PLL	50	50.000000	0.100000 : 250.000000
☐ FCLK_CLK2	IO PLL	50	10.000000	0.100000 : 250.000000
☐ FCLK_CLK3	IO PLL	50	10.000000	0.100000 : 250.000000
> System Debug Clocks				
> Timers				

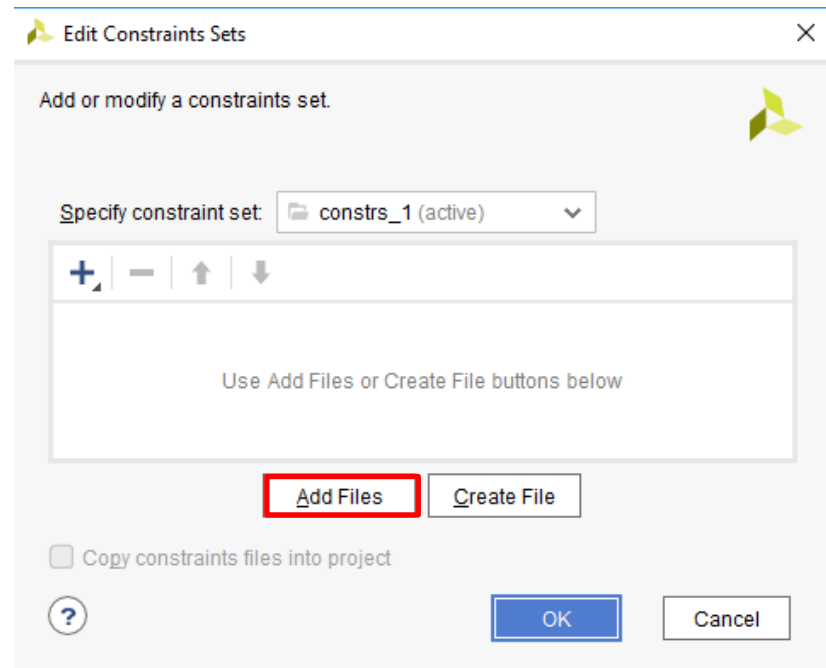
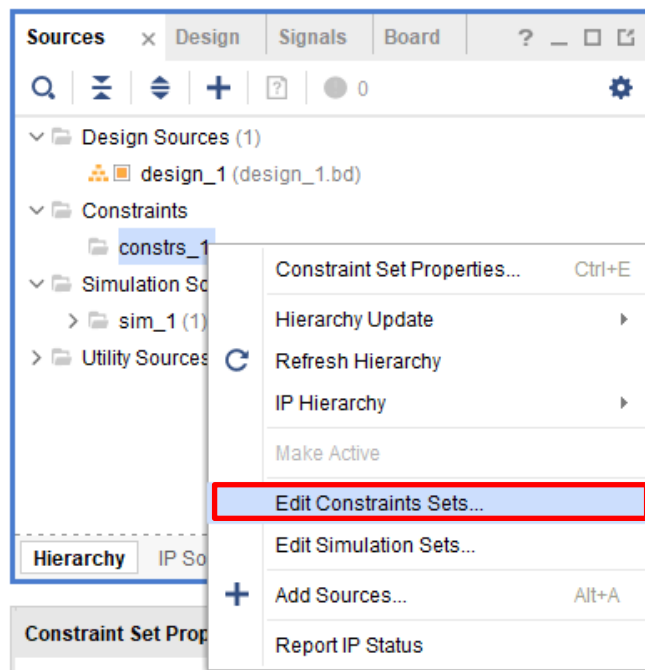
OK Cancel

Zynq SPI Peripheral - DAC. The *FCLK_CLK1* port of the Zynq PS is manually connected to the *DA2clk* port of the *DA2Pmod* IP.

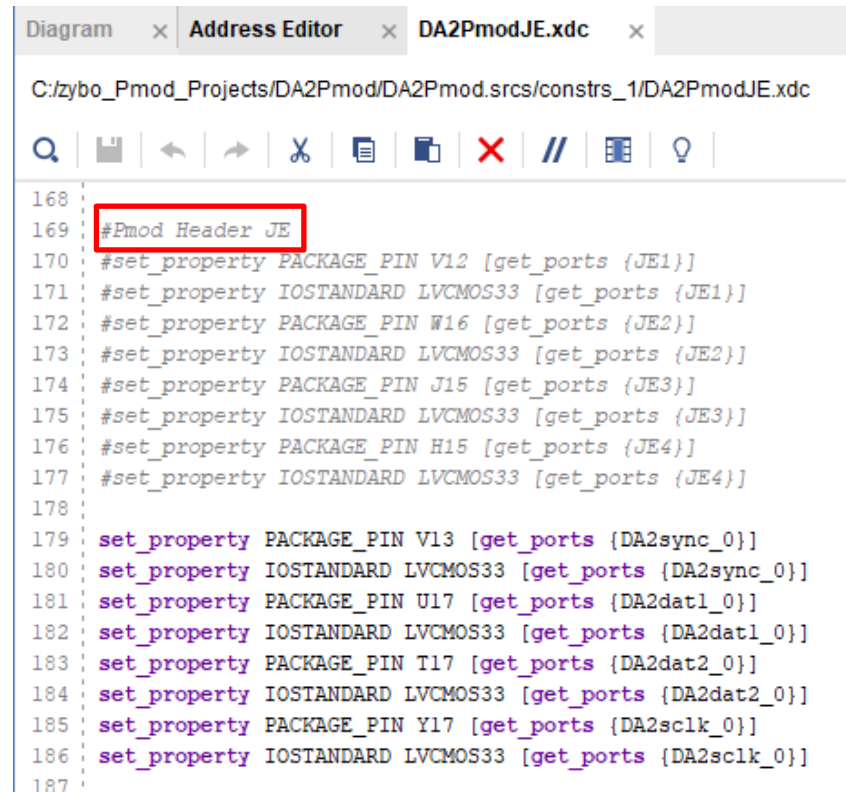
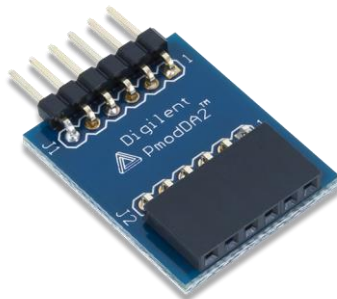
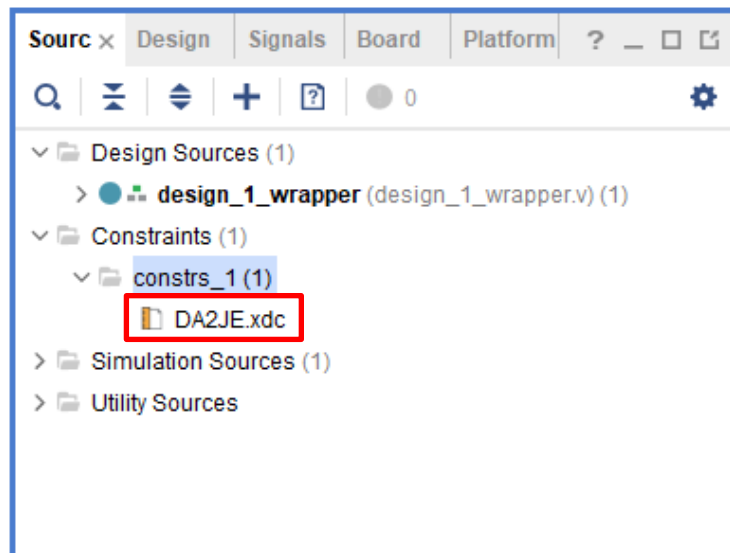


Zynq SPI Peripheral - DAC. An *.xdc* constraint file must be created for this DA2Pmod IP. Copy the *DA2JE.xdc* file to the default directory.

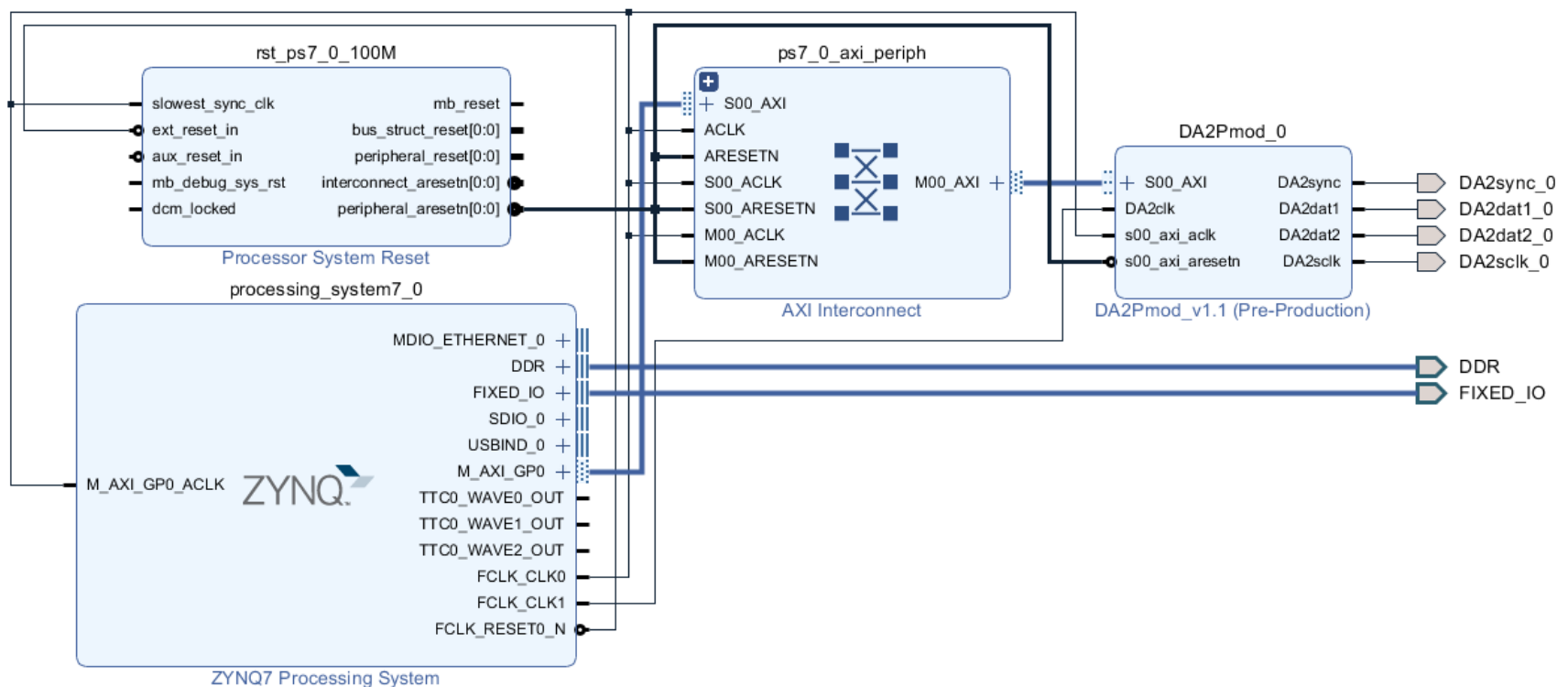
Right-click on *constrs_1* and select *Edit Constraints Sets*.
Select Add Files.



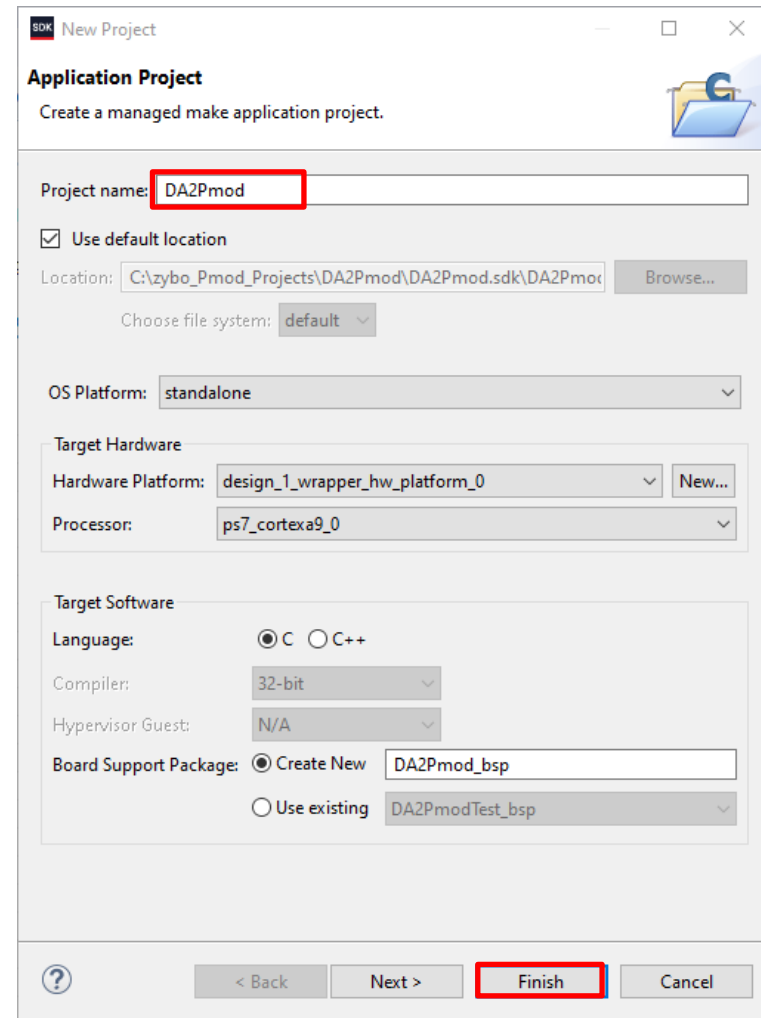
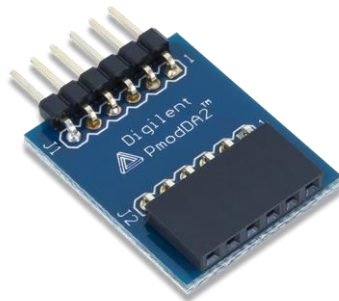
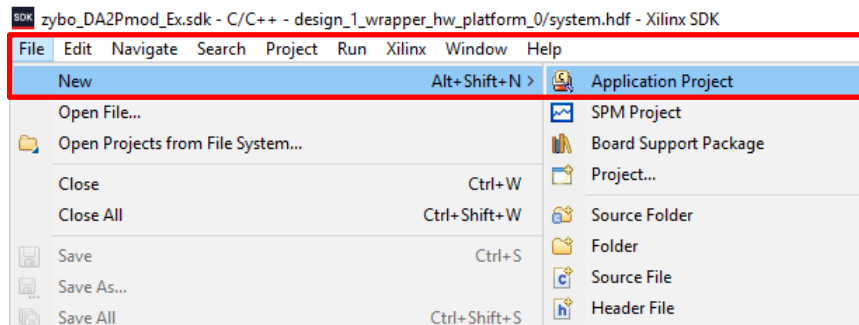
Zynq SPI Peripheral - DAC. The *DA2PmodJE7.xdc* constraint file connects the *DA2Pmod* to Zybo port JE, signal pins 7 to 10. JE pins 11 and 12 are ground and Vcc.



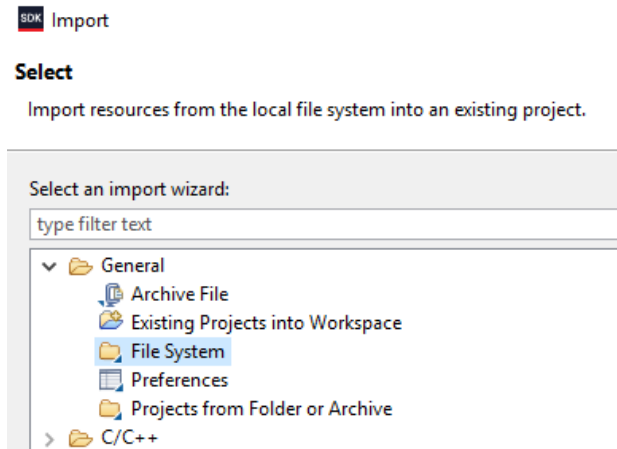
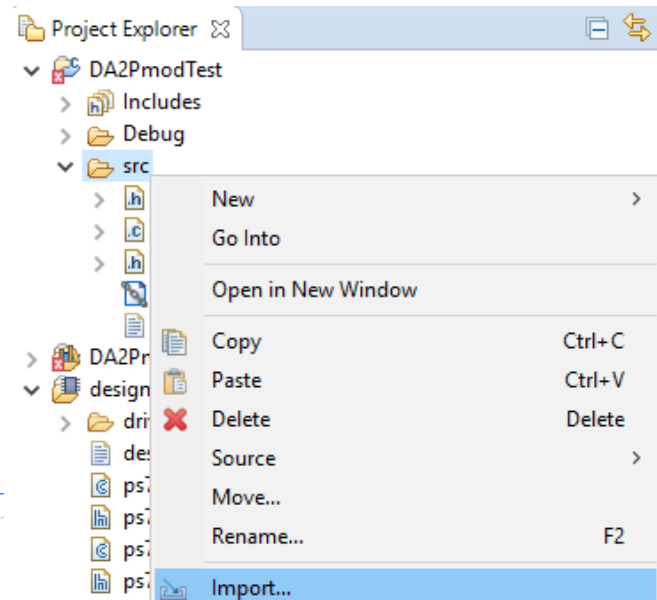
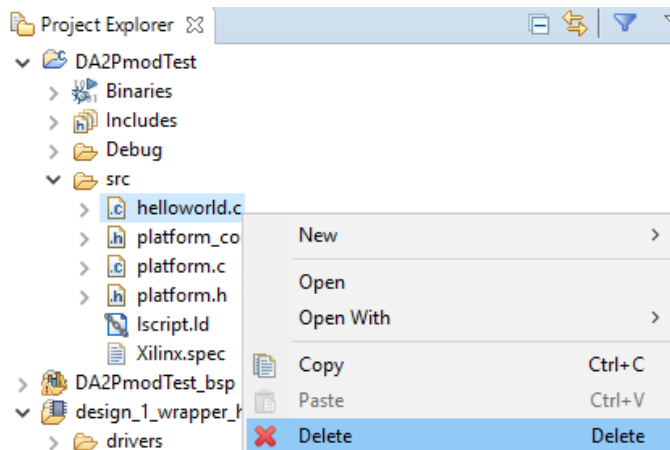
Zynq SPI Peripheral - DAC. Finish the Vivado hardware design by creating the HDL wrapper, generating the bitstream, exporting the hardware (including the bitstream) and launch SDK.



Zynq SPI Peripheral - DAC. In SDK, select *File...New...Application Project* as *DA2Pmod*. Finish to continue.



Zynq SPI Peripheral - DAC. Right-click and delete *helloworld.c*. Right-click on *src...Import*, select *File System* and browse to the location of *DA2Pmod.c* and select.



Zynq SPI Peripheral - DAC. The DA2Pmod base addresses are in the Address Editor in Vivado and are 32-bit registers (4 bytes).

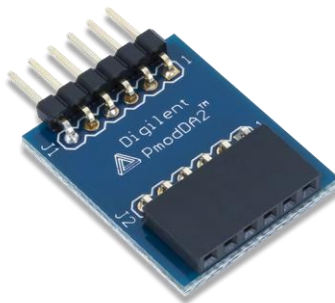
The screenshot displays the Vivado IDE interface. On the left, the 'main.c' file is open, showing C code for a DAC2Pmod test. The code includes headers for xparameters, xil_io, and xil_printf. It defines several memory addresses for DA2 acquisition, data availability, and channel data. The main function initializes variables and prints a message.

On the right, the 'Address Editor' window is open, showing the 'processing_system7_0' block. The 'Data' table lists the addresses for the DAC2Pmod registers.

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
S00_AXI	S00_AXI_r...	0x43C0_0000	6...	0x43C0_FFFF	

Zynq SPI Peripheral – DAC. *DA2Pmod.c* with hardware *handshaking*: DA2acq starts the DAC data output and DA2dav data available signals when completed.

The unipolar DAC signals are outputted.



```
// main.c DA2PmodTest  ECE3622  Dennis Silage  c2019

#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"

//DAC2Pmod from Address Editor in Vivado
#define DA2acq  0x43C00000  //DA2 acquisition - output
#define DA2dav  0x43C00004  //DA2 data available - input
#define DA2dat1 0x43C00008  //DA2 channel 1 data - output
#define DA2dat2 0x43C0000C  //DA2 channel 2 data - output

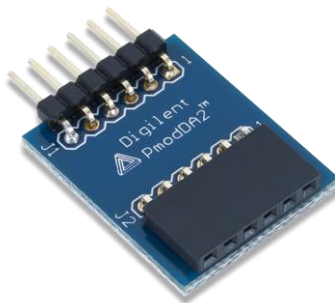
int main(void)
{
    int dacdata=0; //DAC ramp data
    int dacdav;    //DAC data available
    int dacacq=0;  //DAC acquire

    xil_printf("\n\rStarting DA2 Pmod demo test...\n\r");
    Xil_Out32(DA2acq,0);

    while(1)
    {
        if(dacdata==4096)
            dacdata=0;

        dacdav=Xil_In32(DA2dav);
        if(dacdav==0 && dacacq==0) //DAC not in use?
        {
            Xil_Out32(DA2dat1, dacdata); //output DAC data
            Xil_Out32(DA2dat2, dacdata); //output DAC data
            Xil_Out32(DA2acq,1);         //DAC acquire
            dacacq=1;
        }
        if(dacdav==1 && dacacq==1) //DAC data available
        {
            Xil_Out32(DA2acq,0);        //stop DAC acquire
            dacacq=0;
            dacdata++;                  //increment DAC data
        }
    }
}
```

Zynq SPI Peripheral – DAC. *DA2Pmod.c*
 generates a linear ramp
 of data outputted by the
 PmodDA2 DAC.



```
// main.c DA2PmodTest  ECE3622  Dennis Silage  c2019

#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"

//DAC2Pmod from Address Editor in Vivado
#define DA2acq  0x43C00000  //DA2 acquisition - output
#define DA2dav  0x43C00004  //DA2 data available - input
#define DA2dat1 0x43C00008  //DA2 channel 1 data - output
#define DA2dat2 0x43C0000C  //DA2 channel 2 data - output

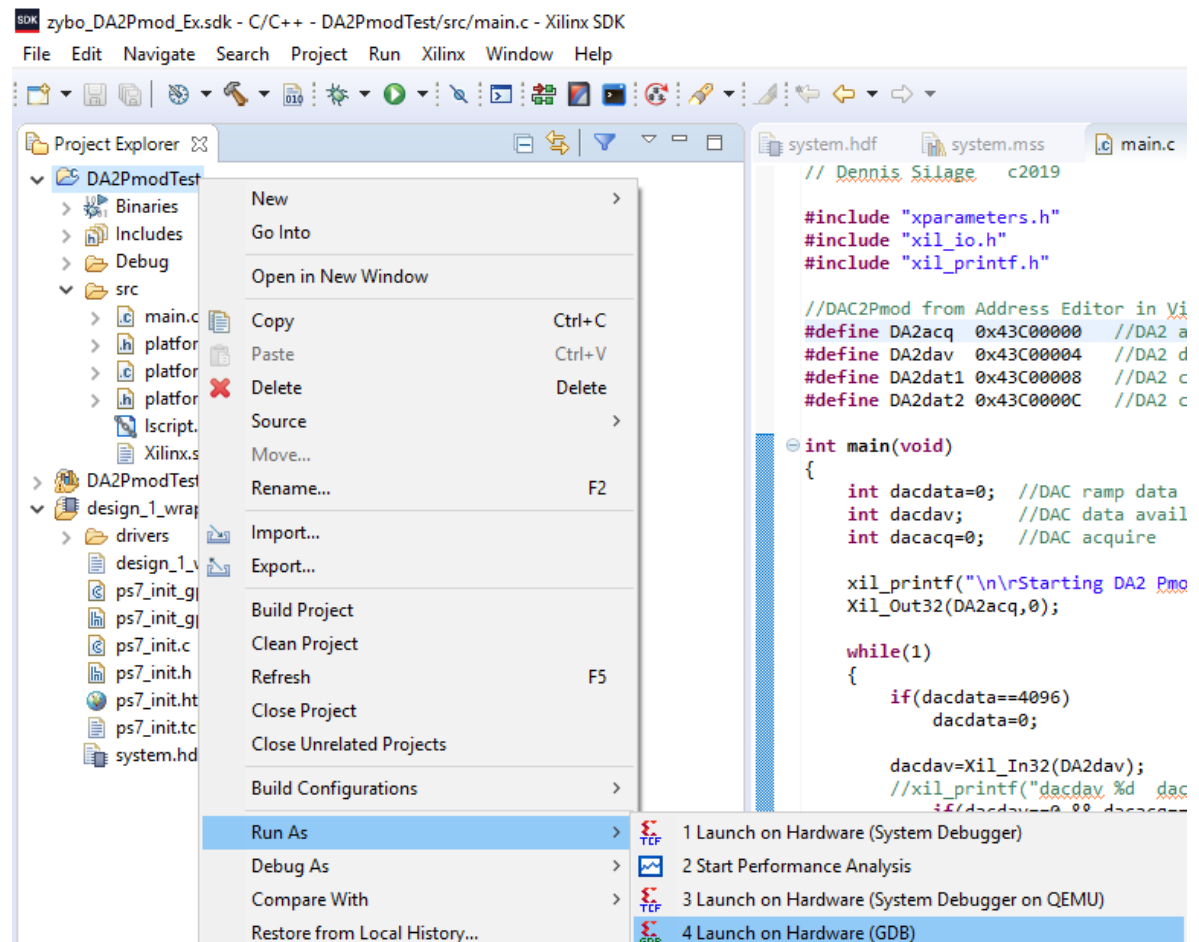
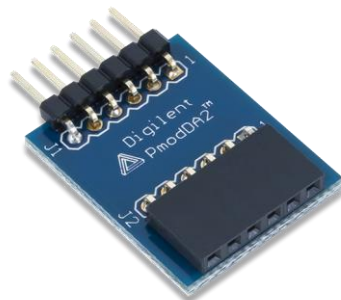
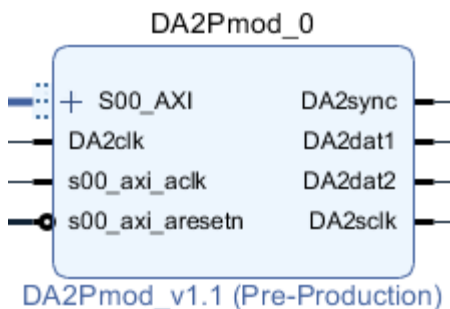
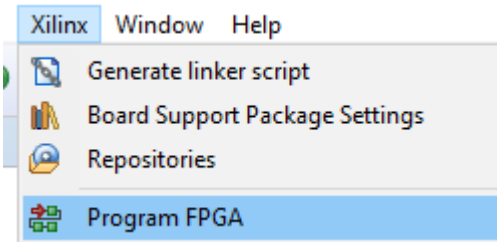
int main(void)
{
    int dacdata=0; //DAC ramp data
    int dacdav;    //DAC data available
    int dacacq=0;  //DAC acquire

    xil_printf("\n\rStarting DA2 Pmod demo test...\n\r");
    Xil_Out32(DA2acq,0);

    while(1)
    {
        if(dacdata==4096)
            dacdata=0;

        dacdav=Xil_In32(DA2dav);
        if(dacdav==0 && dacacq==0) //DAC not in use?
        {
            Xil_Out32(DA2dat1, dacdata); //output DAC data
            Xil_Out32(DA2dat2, dacdata); //output DAC data
            Xil_Out32(DA2acq,1);         //DAC acquire
            dacacq=1;
        }
        if(dacdav==1 && dacacq==1) //DAC data available
        {
            Xil_Out32(DA2acq,0);         //stop DAC acquire
            dacacq=0;
            dacdata++;                    //increment DAC data
        }
    }
}
```


Zynq SPI Peripheral - DAC. Program the FPGA and *Run As...4 Launch on Hardware (GDB)*.



Zynq SPI Peripheral - DAC. Connect the Waveforms oscilloscope channel 1 input to the PmodDA2 channel 1 output (pin 1) and the Waveforms oscilloscope ground to PmodDA2 pin 5.



Zynq SPI Peripheral - DAC. The DA2Pmod project outputs a linear ramp at full resolution of 4096 points in 7.662 msec or $\approx 1.870 \mu\text{sec/sample}$.



End of Zynq

SPI Peripherals:

ADC and DAC

